

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikrokontroler

Fito Dwi Ardiansah - 5024241053

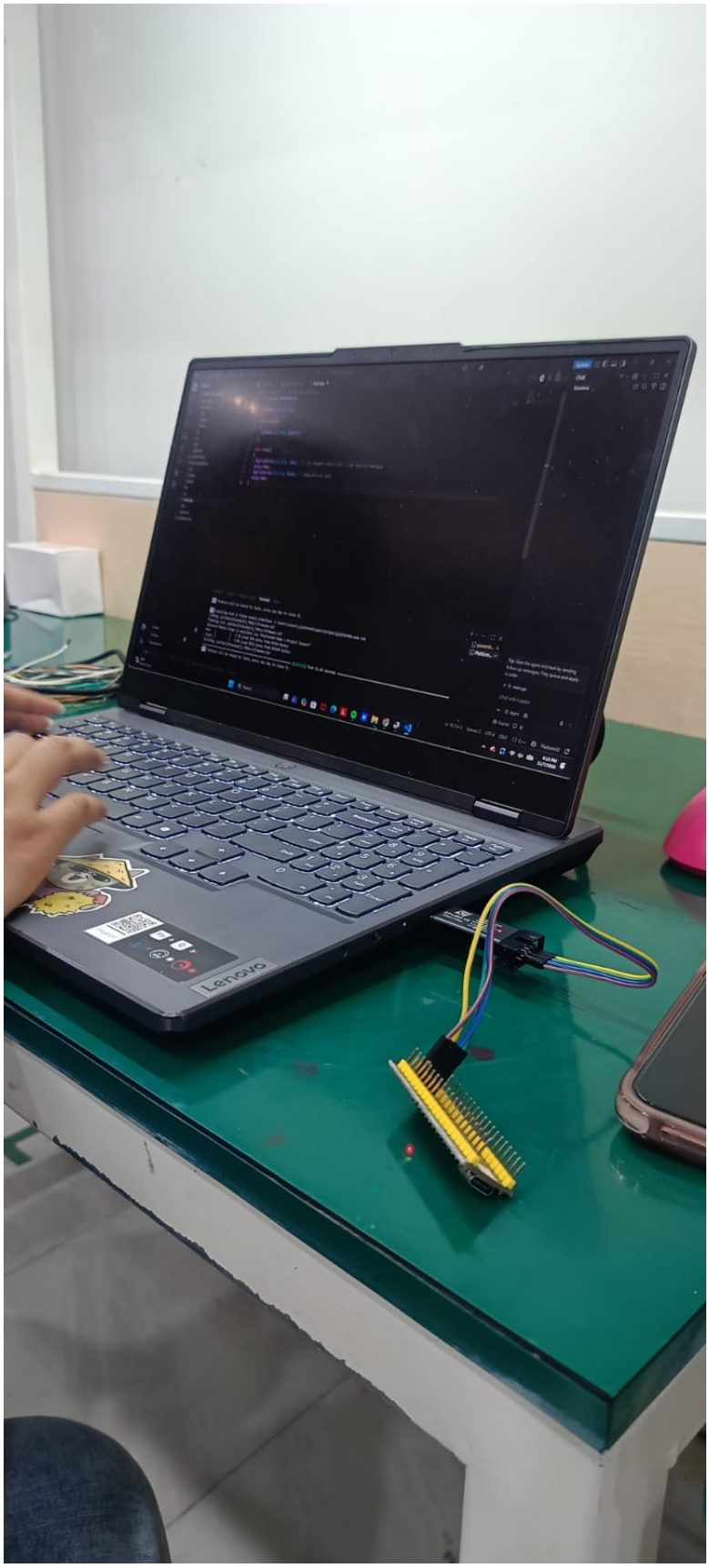
2026

1 Langkah-Langkah Percobaan

Sebelum memulai seluruh percobaan, dibuat satu project PlatformIO untuk Modul 1 ini. Setiap percobaan hanya mengganti isi platformio.ini dan isi src/main.cpp atau src/main.c sesuai kebutuhan masing-masing percobaan, tanpa membuat project baru.

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Board STM32 Blackpill diperiksa terlebih dahulu sebelum dihubungkan ke ST-Link.



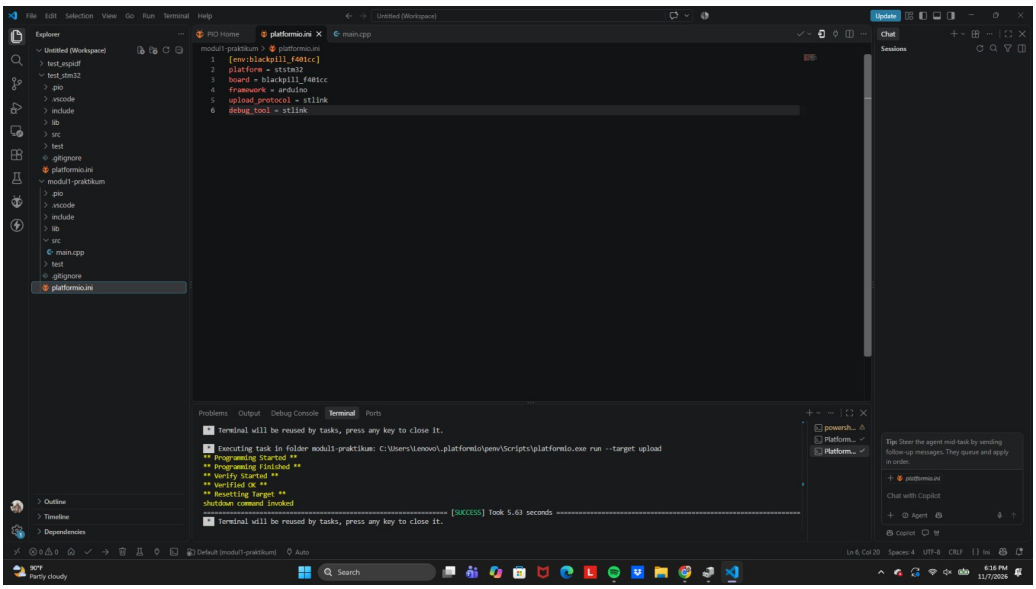
Gambar 1: Board STM32 Blackpill sebelum dihubungkan ke ST-Link

ST-Link kemudian dihubungkan ke board Blackpill sesuai pasangan pin SWDIO, SWCLK, GND, dan 3.3V, lalu dicolokkan ke komputer lewat USB.



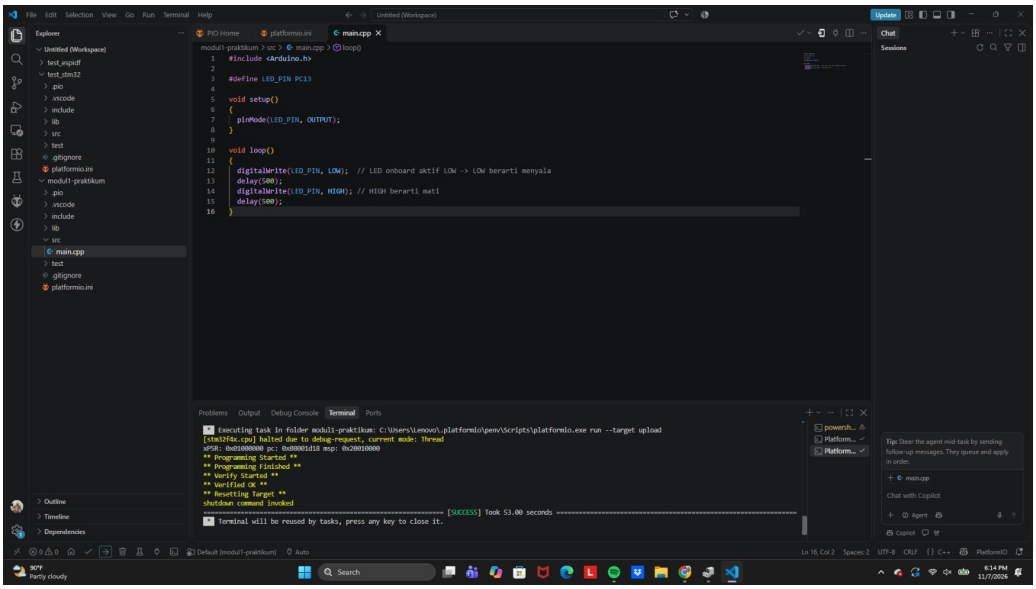
Gambar 2: Koneksi ST-Link dengan board STM32 Blackpill

Isi platformio.ini diatur dengan platform ststm32, board sesuai chip yang digunakan, framework arduino, upload_protocol dan debug_tool stlink.



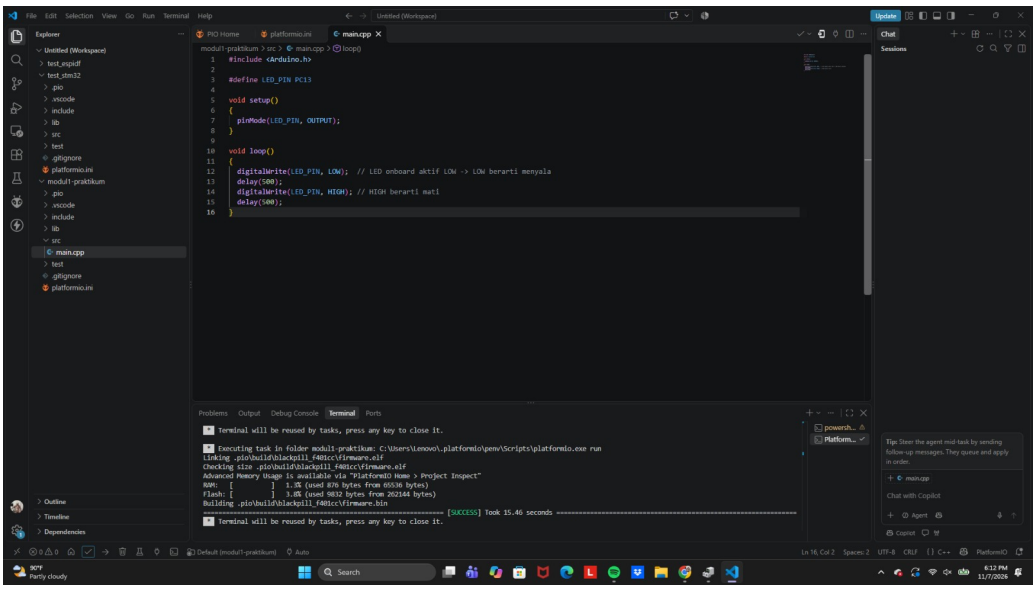
Gambar 3: Konfigurasi platformio.ini untuk STM32 Blackpill

Kode Blink ditulis pada src/main.cpp menggunakan LED onboard di pin PC13, lalu dilakukan Build dan Upload.



Gambar 4: Kode program Blink pada STM32 Blackpill

Proses Build menunjukkan status SUCCESS.



Gambar 5: Hasil Build program Blink STM32 menunjukkan status SUCCESS

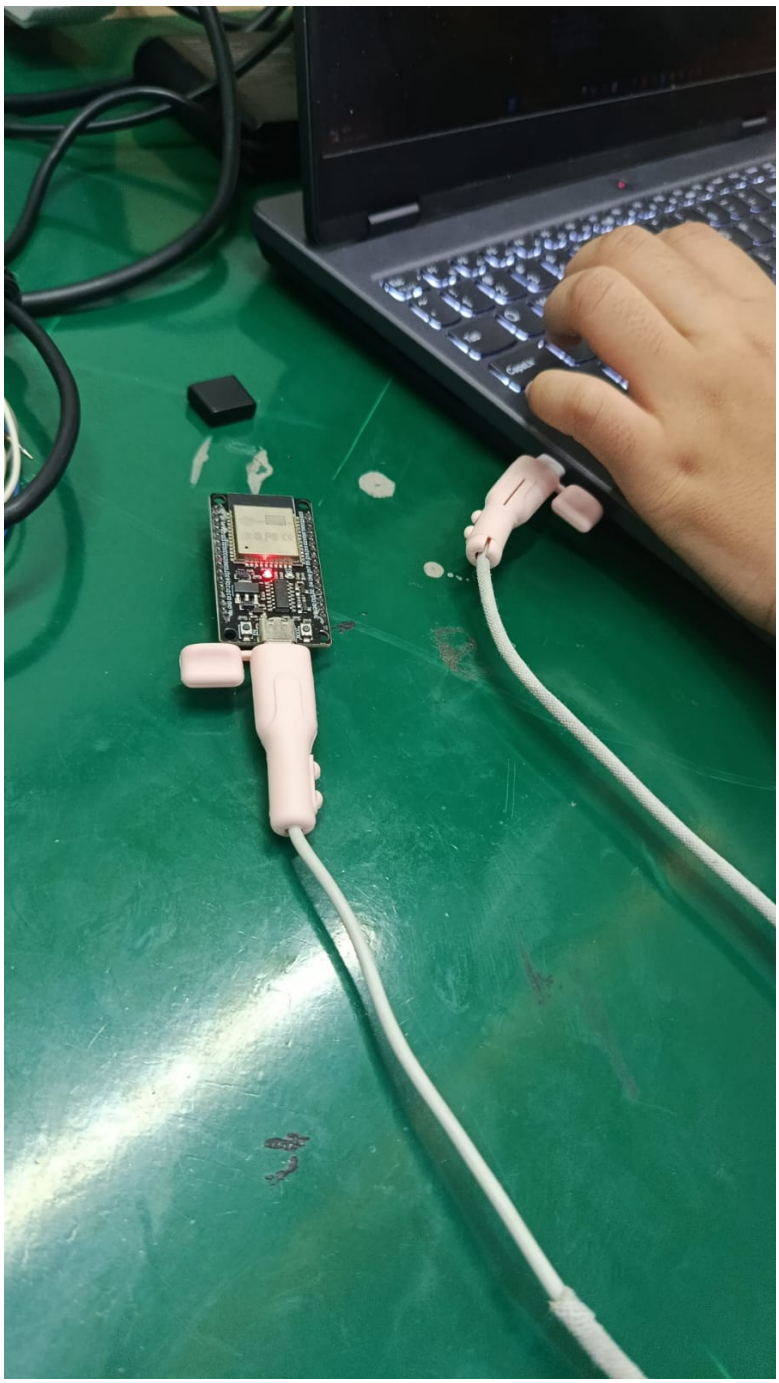
Setelah program ter-upload, LED pada board berkedip nyala-mati tiap setengah detik.



Gambar 6: LED onboard STM32 Blackpill menyala setelah program di-upload

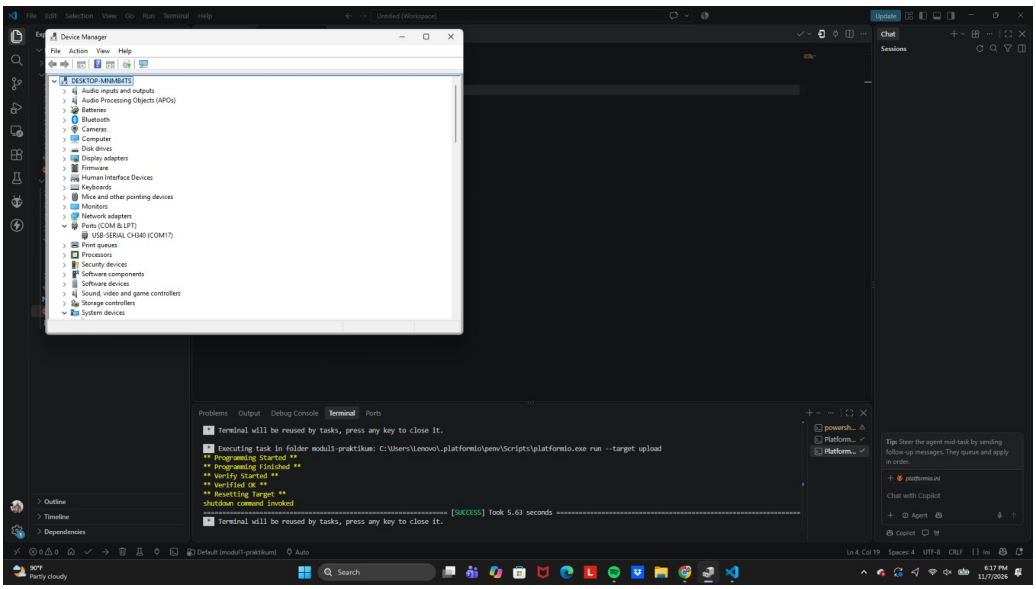
1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

ESP32 dicolokkan ke komputer lewat USB.



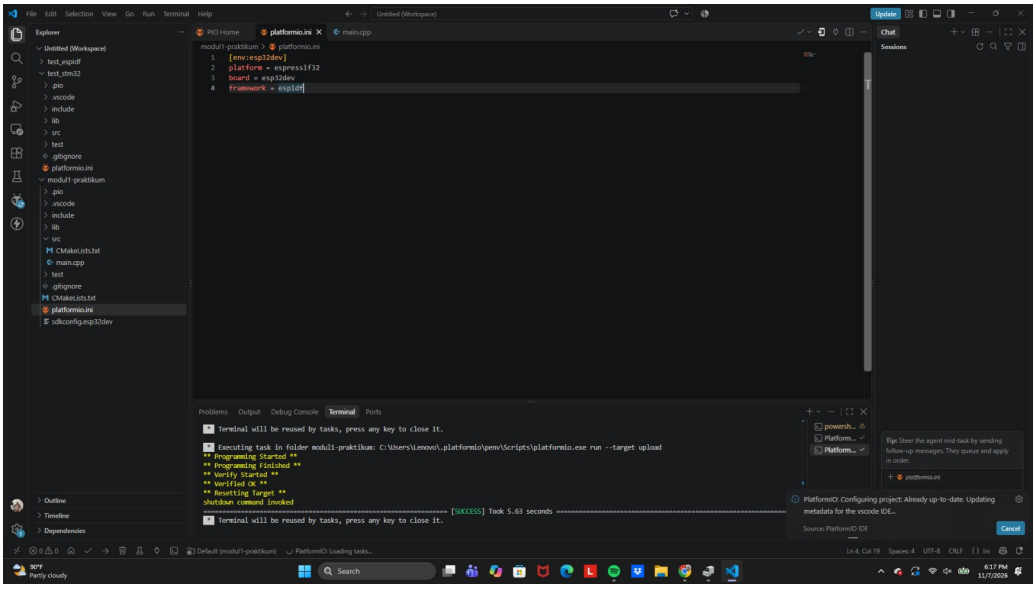
Gambar 7: Board ESP32 terhubung ke komputer melalui USB

Port USB-to-Serial board diperiksa melalui Device Manager untuk memastikan board terdeteksi dengan baik.



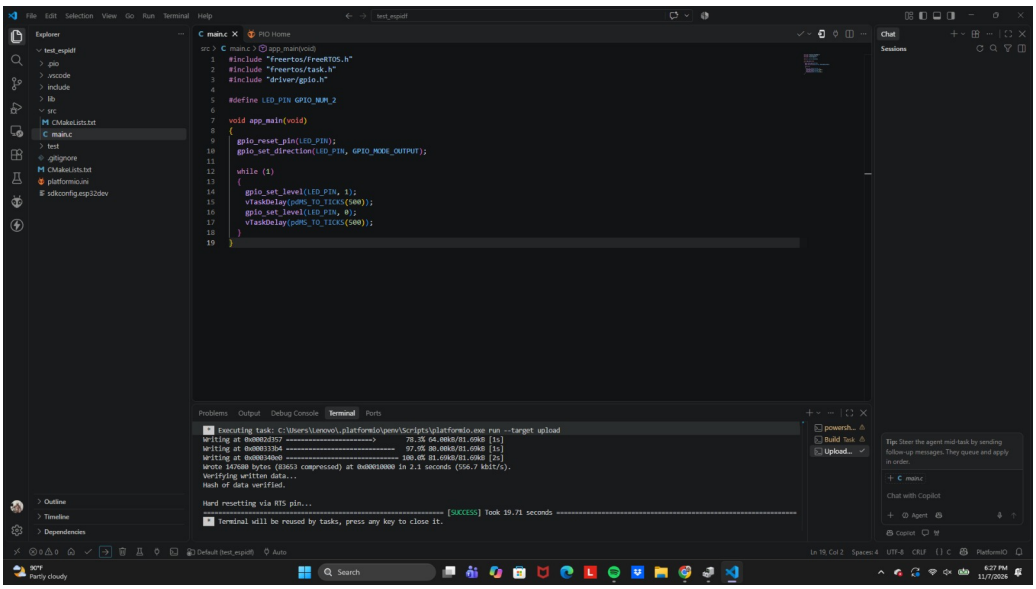
Gambar 8: Verifikasi port ESP32 pada Device Manager

Isi platformio.ini diganti menjadi platform espressif32, board esp32dev, framework espidf.



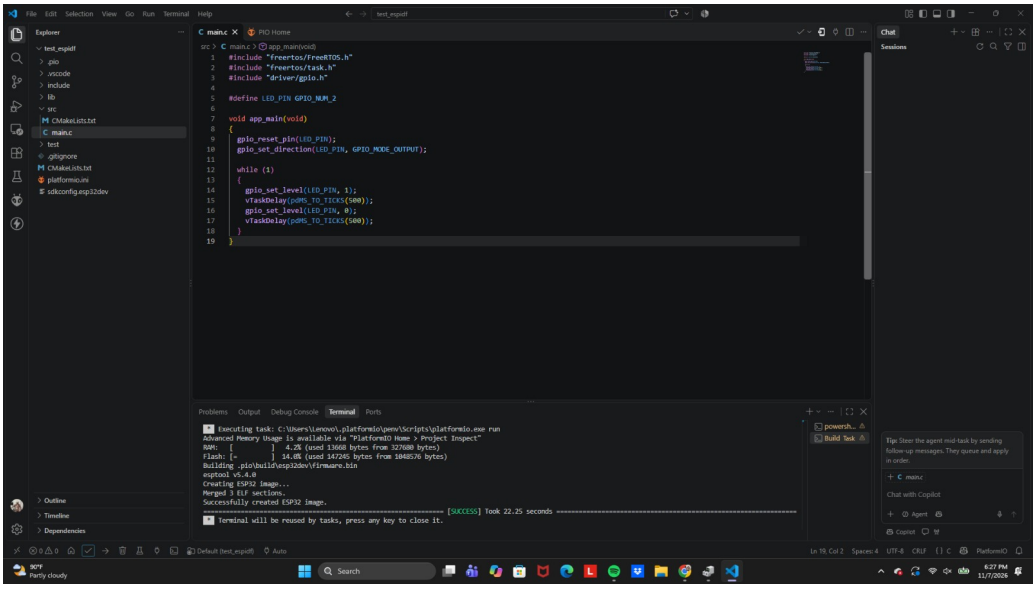
Gambar 9: Konfigurasi platformio.ini dengan framework espidf

File src/main.cpp dihapus, dibuat file baru src/main.c berisi kode Blink menggunakan struktur app_main() khas ESP-IDF, lalu dilakukan Build dan Upload.



Gambar 10: Kode program Blink ESP32 menggunakan framework ESP-IDF

Proses Build menunjukkan status SUCCESS.



Gambar 11: Hasil Build program Blink ESP32 dengan ESP-IDF menunjukkan status SUCCESS

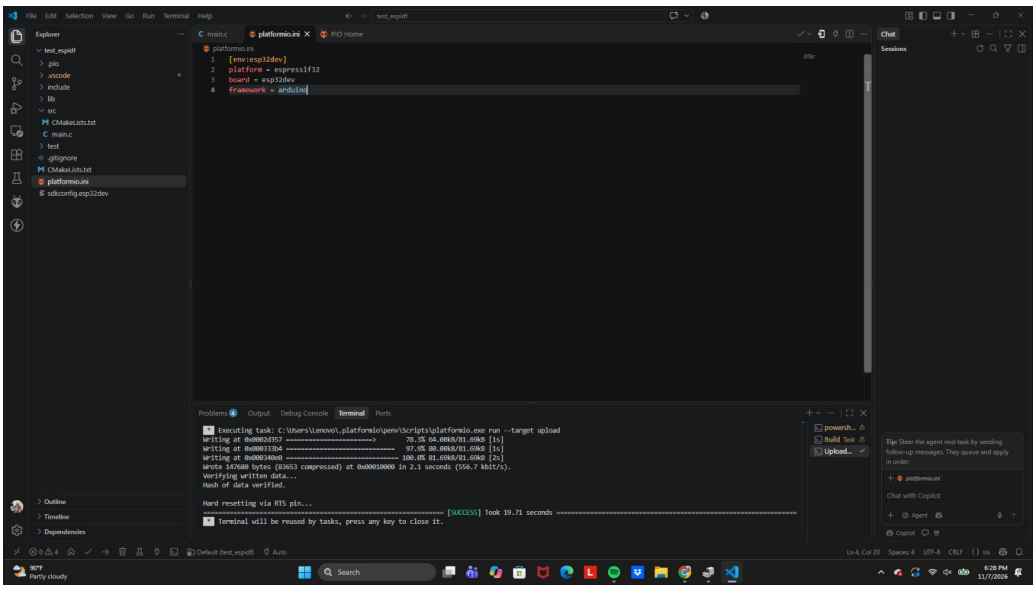
Setelah program ter-upload, LED onboard ESP32 terlihat menyala.



Gambar 12: LED onboard ESP32 menyala setelah program di-upload

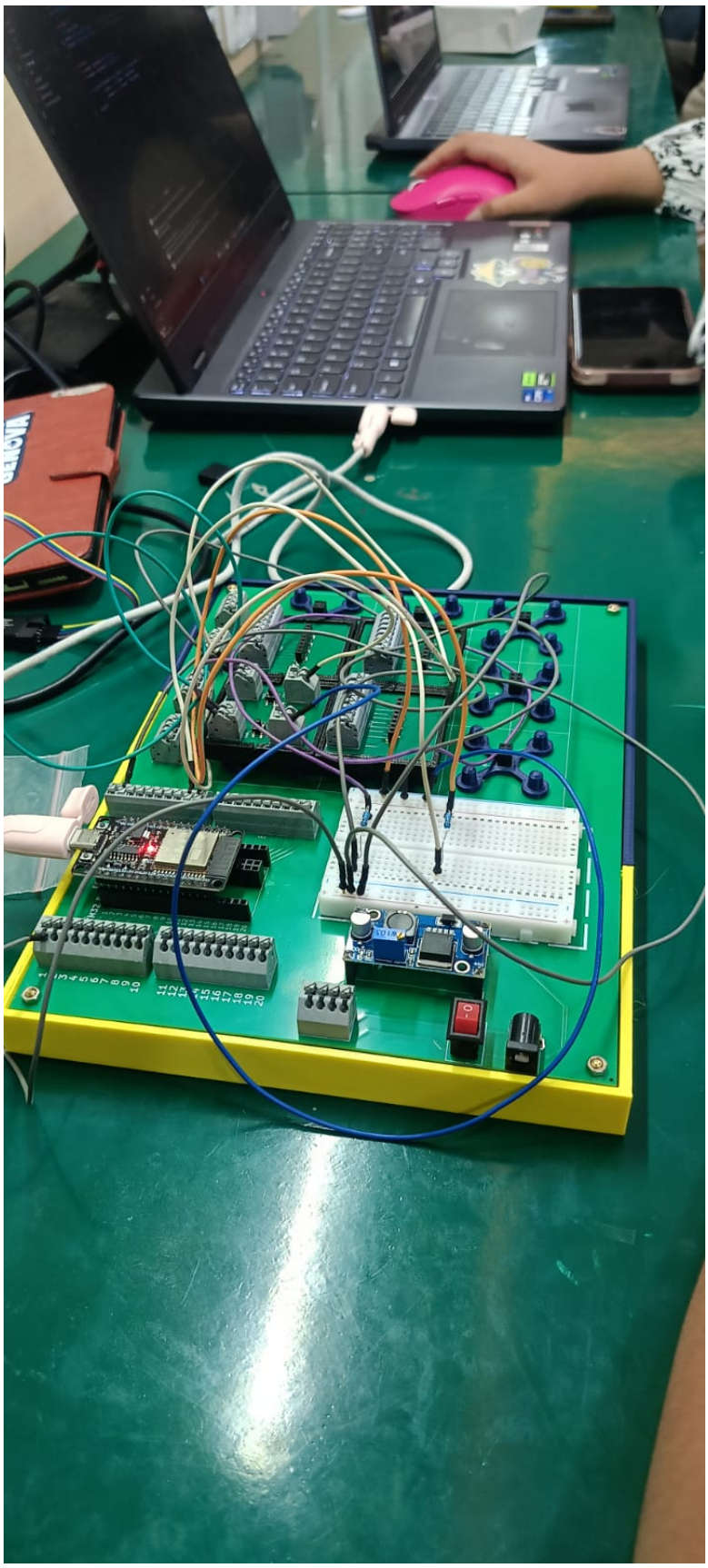
1.3 Percobaan 3: Pull-up dan Pull-down, Eksternal vs Internal

Isi platformio.ini dikembalikan ke framework arduino.

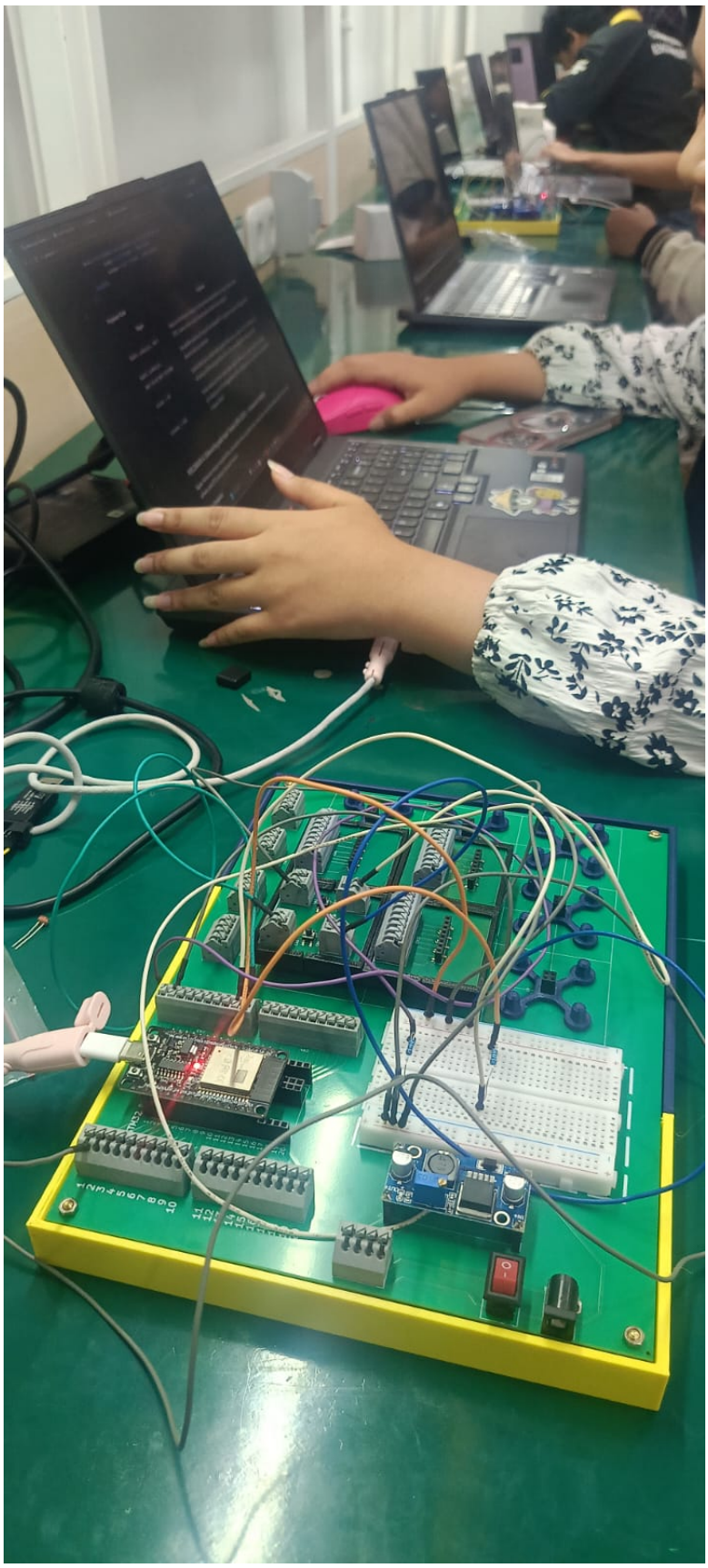


Gambar 13: Konfigurasi platformio.ini dikembalikan ke framework arduino

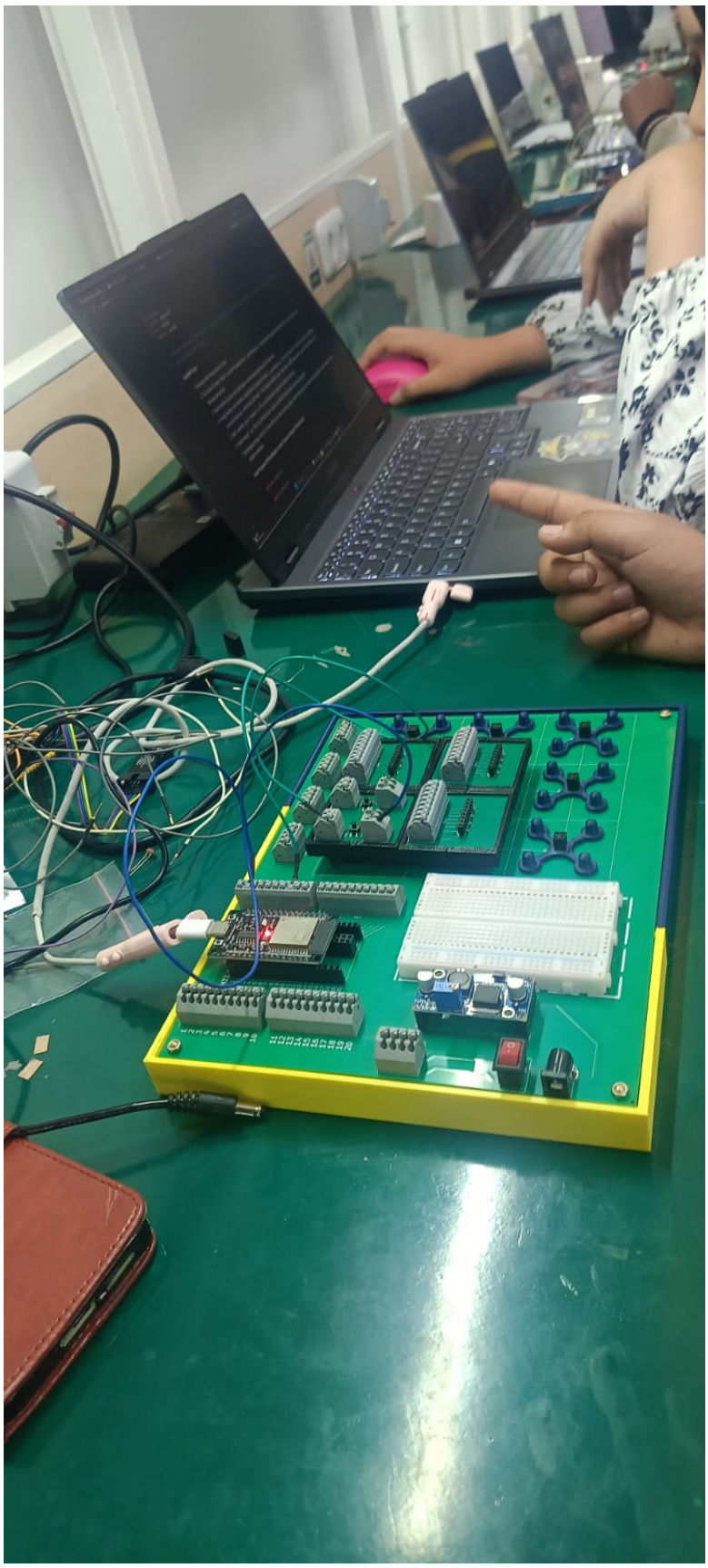
Dua pushbutton dipasang pada GPIO 32 dan GPIO 33. Pada bagian pertama, resistor 10kOhm dipasang secara eksternal, pushbutton 1 sebagai pull-up pada GPIO 32 dan pushbutton 2 sebagai pull-down pada GPIO 33.



Gambar 14: Rangkaian dua pushbutton pada GPIO 32 dan GPIO 33

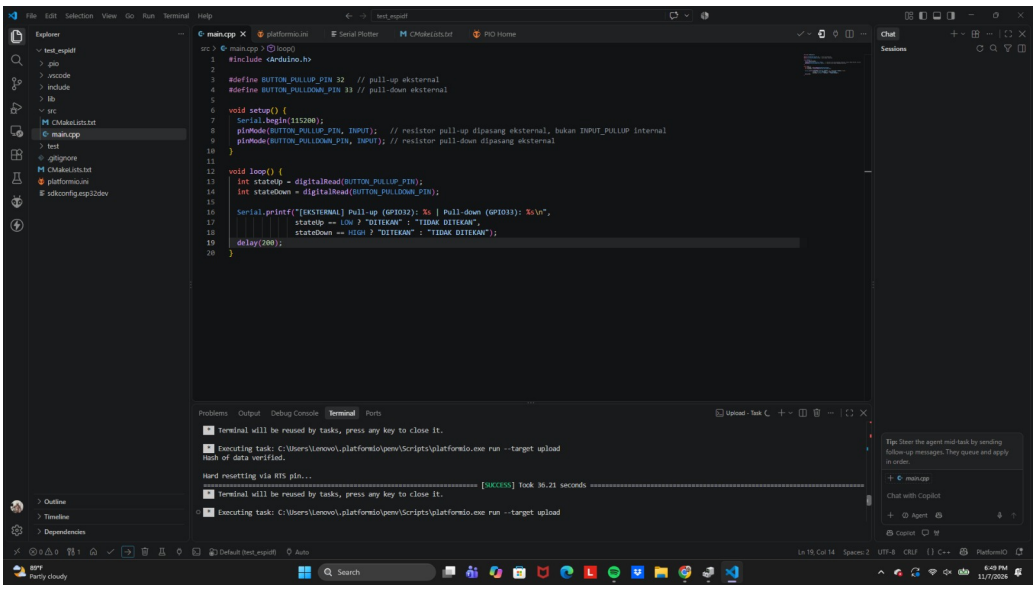


Gambar 15: Suasana pemasangan rangkaian pull-up dan pull-down eksternal

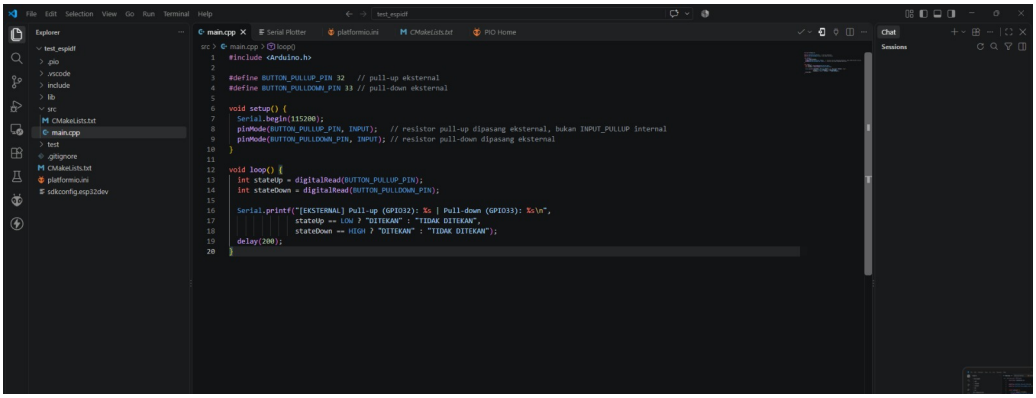


Gambar 16: Pemeriksaan rangkaian sebelum pengujian dimulai

Kode Program A ditulis pada src/main.cpp menggunakan pinMode INPUT biasa karena resistor sudah dipasang eksternal, lalu dilakukan Build dan Upload.

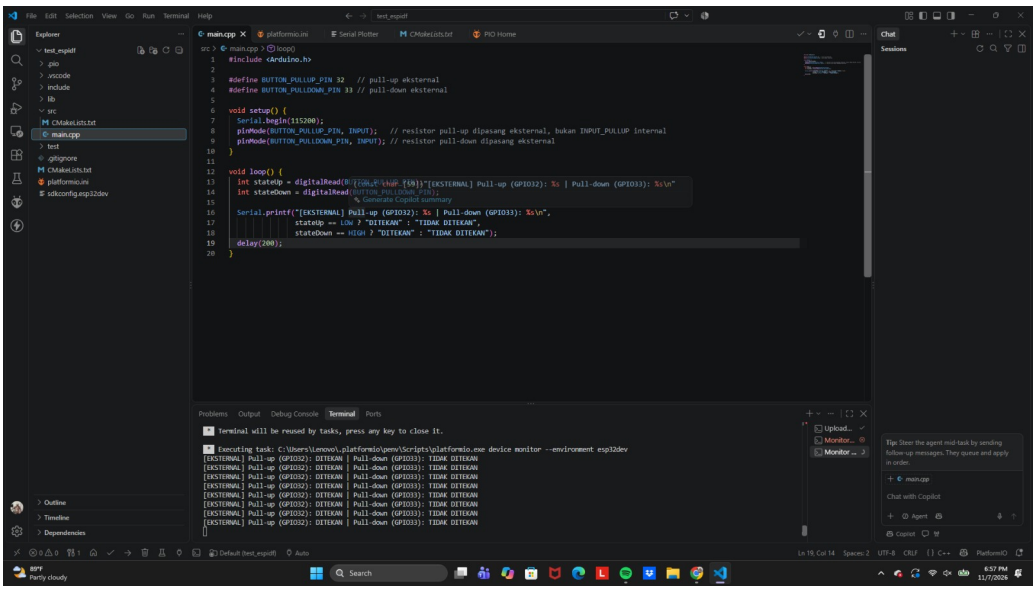


Gambar 17: Kode Program A beserta proses Upload ke board

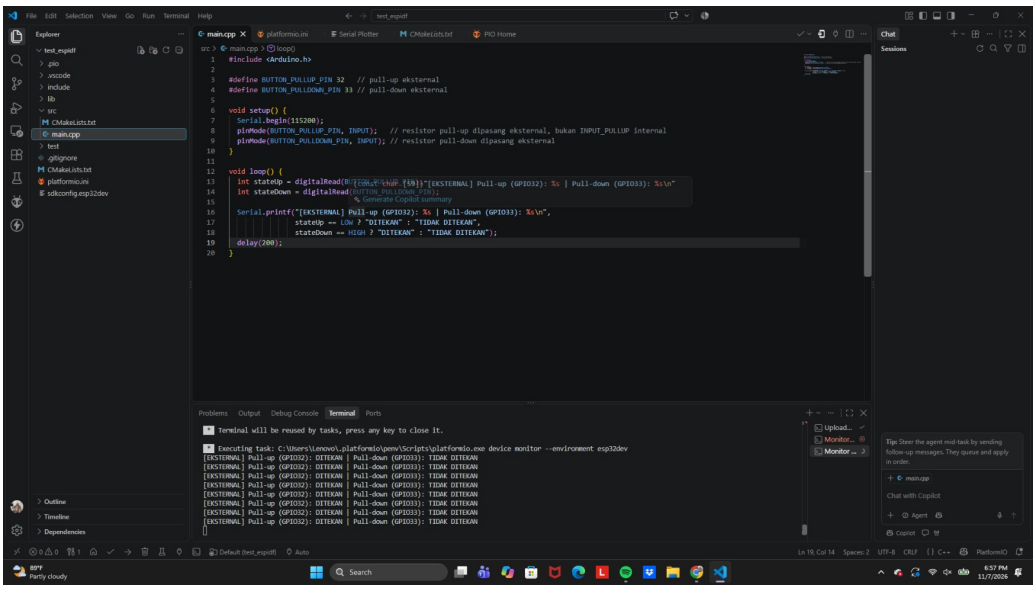


Gambar 18: Kode Program A, pembacaan pull-up dan pull-down eksternal

Kedua tombol ditekan secara bergantian dan hasilnya dicatat dari Serial Monitor.

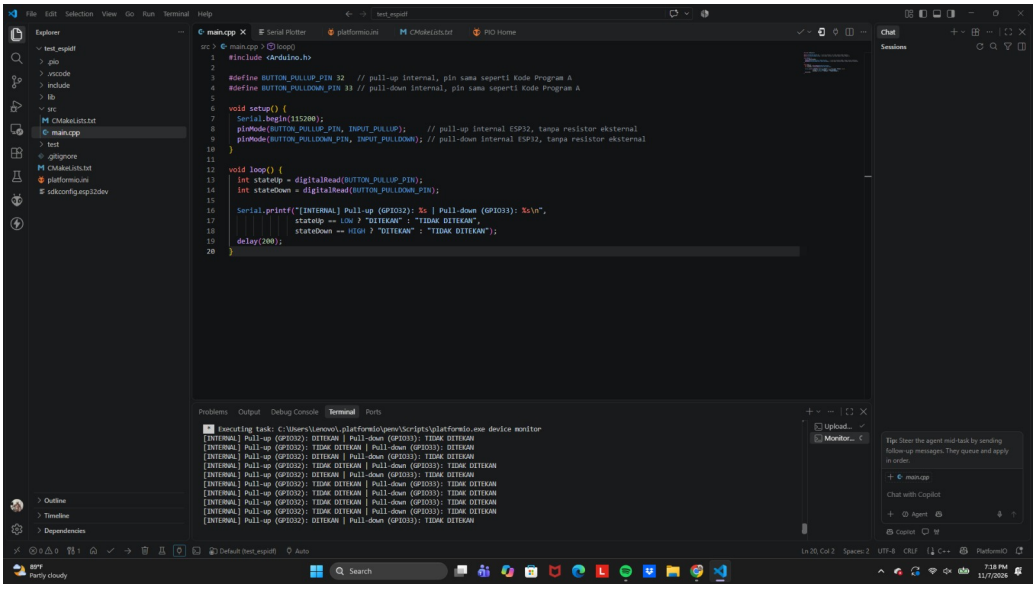


Gambar 19: Hasil pembacaan Serial Monitor pada pengujian pull-up dan pull-down eksternal, cuplikan pertama



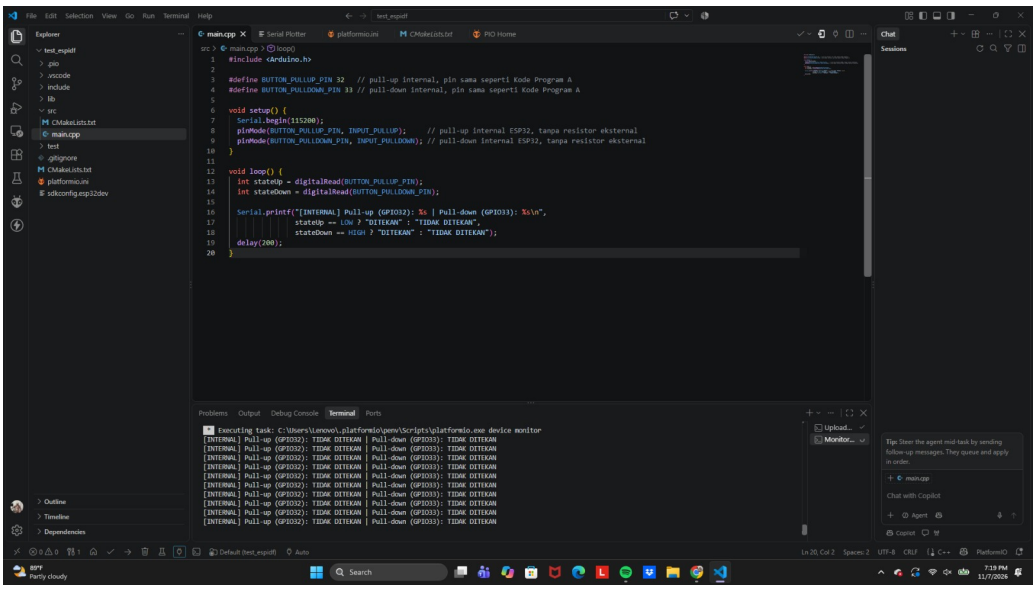
Gambar 20: Hasil pembacaan Serial Monitor pada pengujian pull-up dan pull-down eksternal, cuplikan kedua

Kedua resistor eksternal kemudian dilepas, tombol tetap terpasang pada pin yang sama. Kode program diganti dengan Kode Program B yang menggunakan INPUT_PULLUP dan INPUT_PULLDOWN internal, lalu dilakukan Build dan Upload kembali.



Gambar 21: Kode Program B, pembacaan pull-up dan pull-down internal

Kedua tombol yang sama diuji kembali dan hasilnya dicatat dari Serial Monitor.

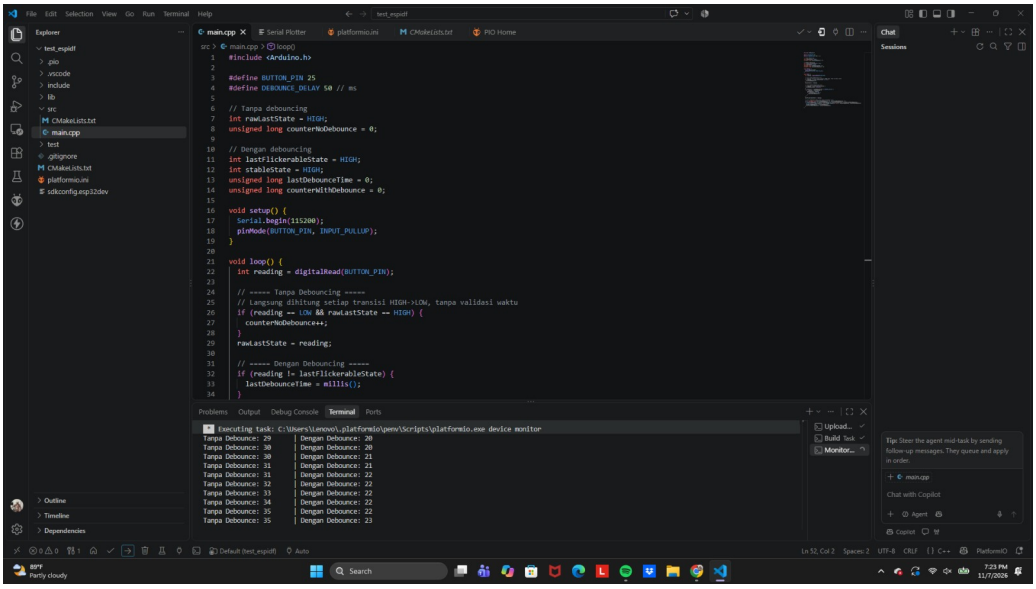


Gambar 22: Hasil pembacaan Serial Monitor pada pengujian pull-up dan pull-down internal

Hasil pembacaan pull-up dan pull-down pada kedua kondisi tersebut menunjukkan pola yang sama, yaitu pull-up bernilai HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan pull-down bernilai LOW saat tidak ditekan dan HIGH saat ditekan.

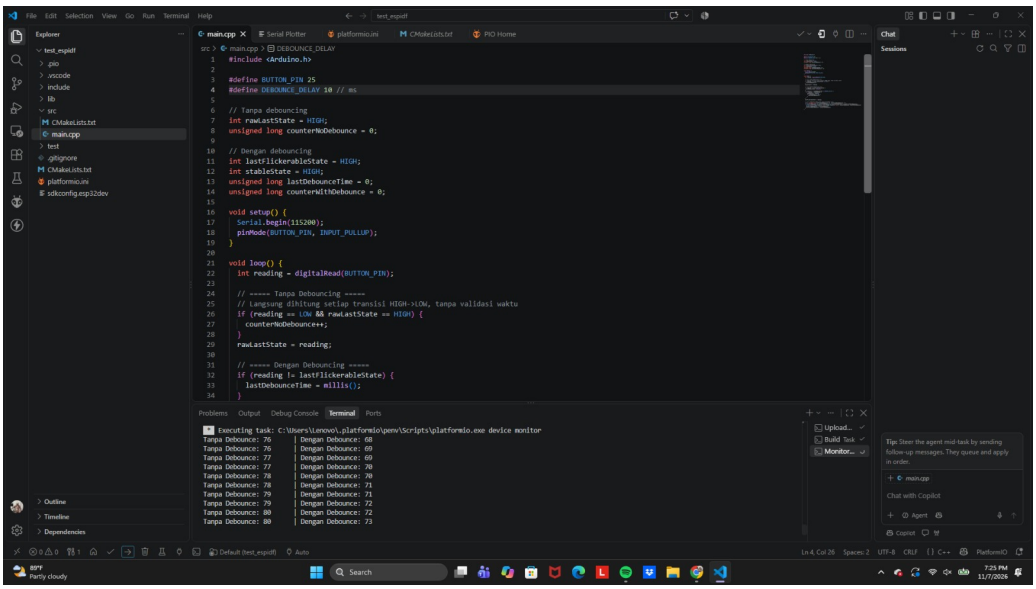
1.4 Percobaan 4: Debouncing pada Input Tombol GPIO

Satu pushbutton dipasang pada GPIO 25 menggunakan pull-up internal. Kode program ditulis dengan dua penghitung, yaitu counterNoDebounce tanpa penyaring getaran tombol dan counterWithDebounce dengan penyaring, menggunakan DEBOUNCE_DELAY awal sebesar 50 milidetik.

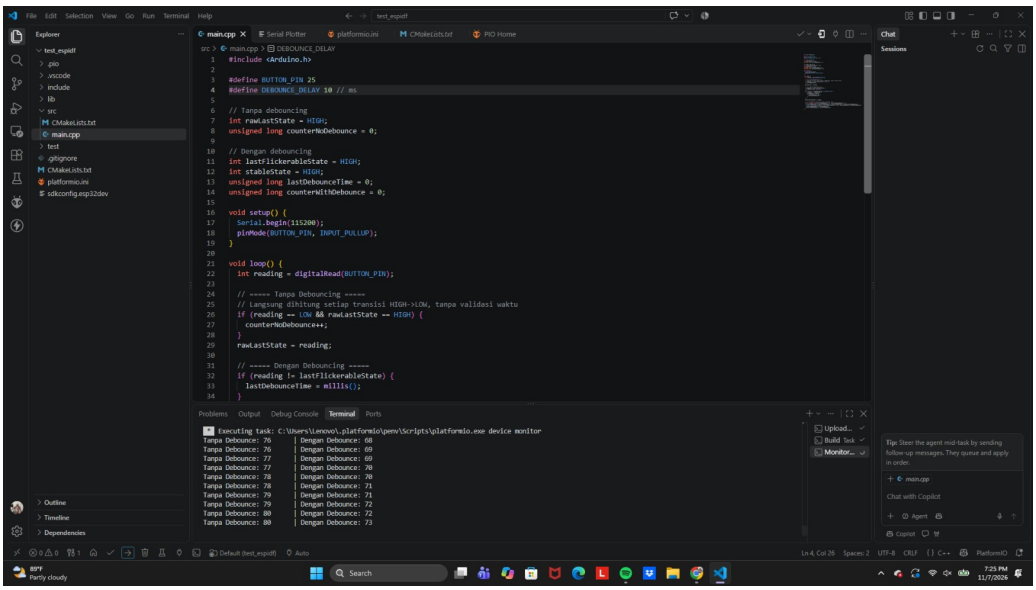


Gambar 23: Kode program debouncing dengan DEBOUNCE_DELAY 50 milidetik

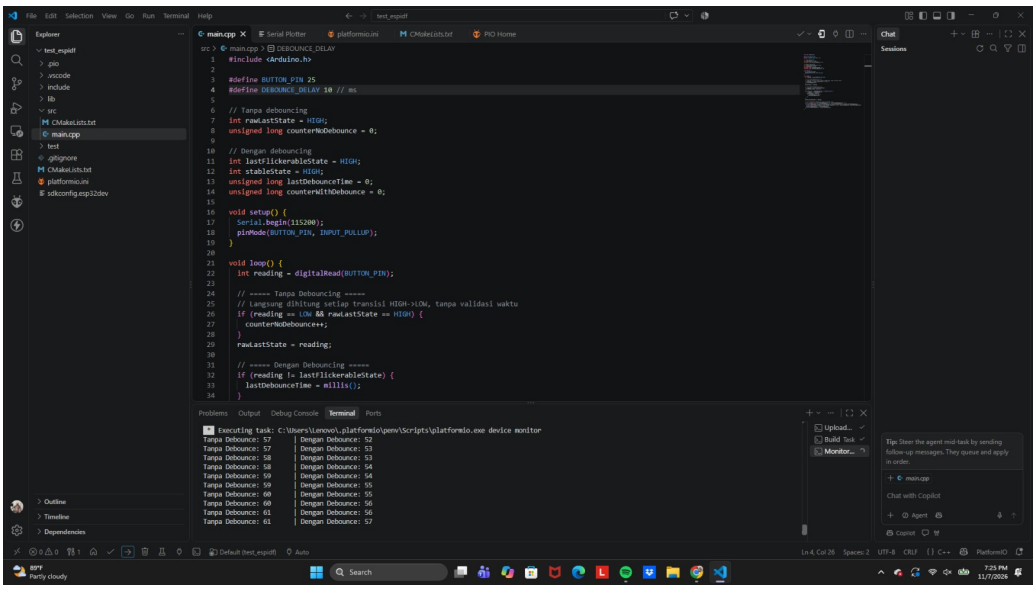
Nilai DEBOUNCE_DELAY kemudian diubah menjadi 10 milidetik, tombol ditekan berulang, dan hasilnya diamati pada beberapa cuplikan Serial Monitor berikut.



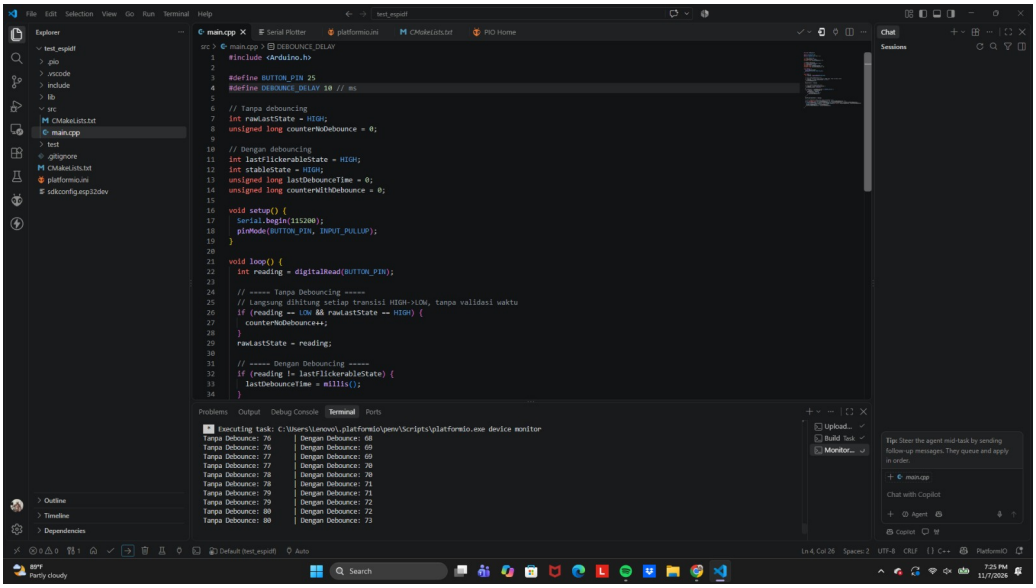
Gambar 24: Hasil Serial Monitor dengan DEBOUNCE_DELAY 10 milidetik, cuplikan 1



Gambar 25: Hasil Serial Monitor dengan DEBOUNCE_DELAY 10 milidetik, cuplikan 2

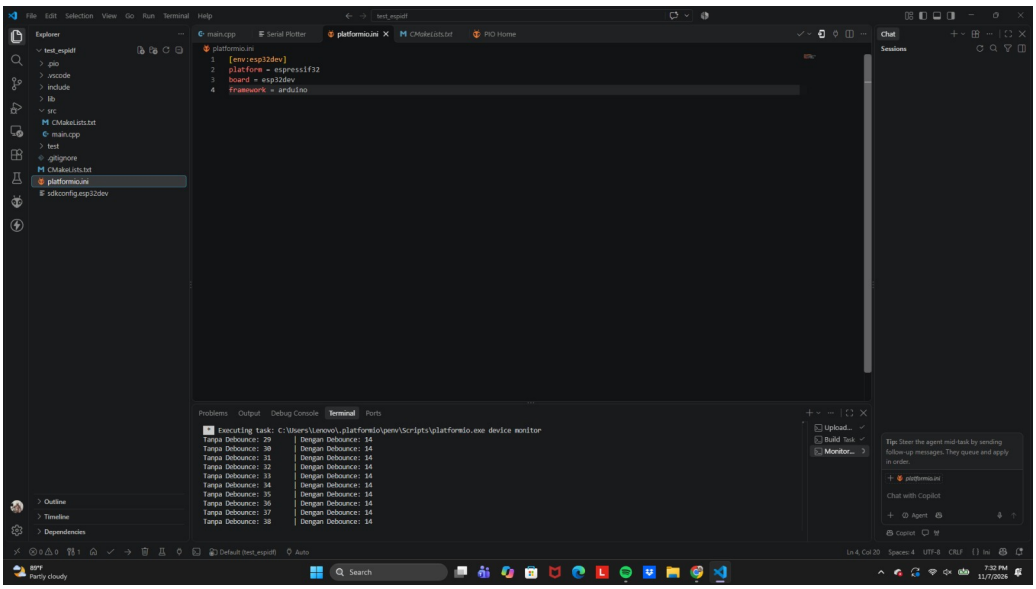


Gambar 26: Hasil Serial Monitor dengan DEBOUNCE_DELAY 10 milidetik, cuplikan 3

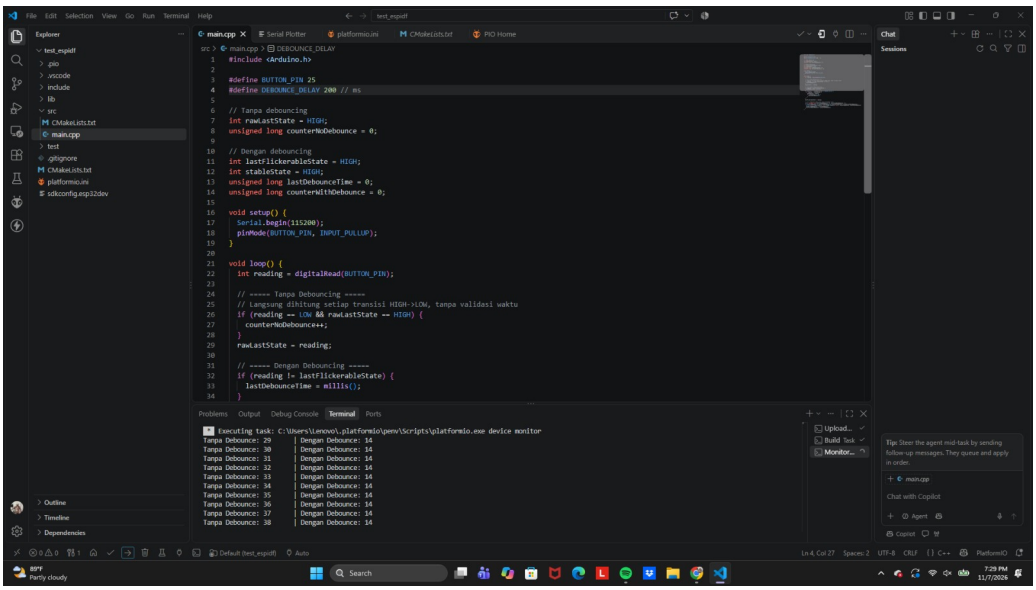


Gambar 27: Hasil Serial Monitor dengan DEBOUNCE_DELAY 10 milidetik, cuplikan 4

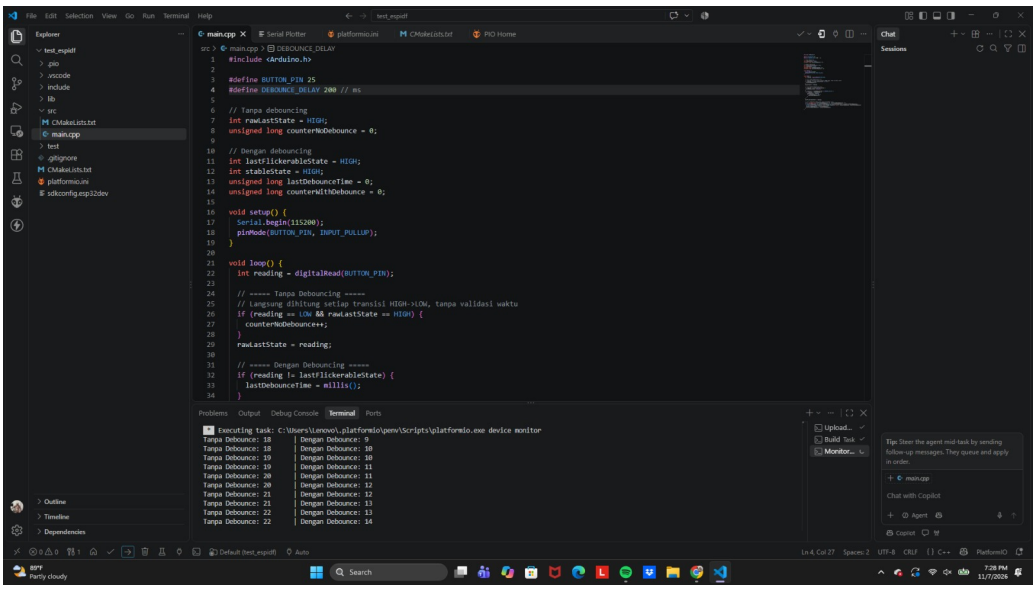
Pengujian diulang kembali dengan mengubah nilai DEBOUNCE_DELAY menjadi 200 milidetik, kemudian tombol ditekan dengan cepat secara berulang.



Gambar 28: Hasil Serial Monitor dengan DEBOUNCE_DELAY 200 milidetik, cuplikan 1



Gambar 29: Hasil Serial Monitor dengan DEBOUNCE_DELAY 200 milidetik, cuplikan 2

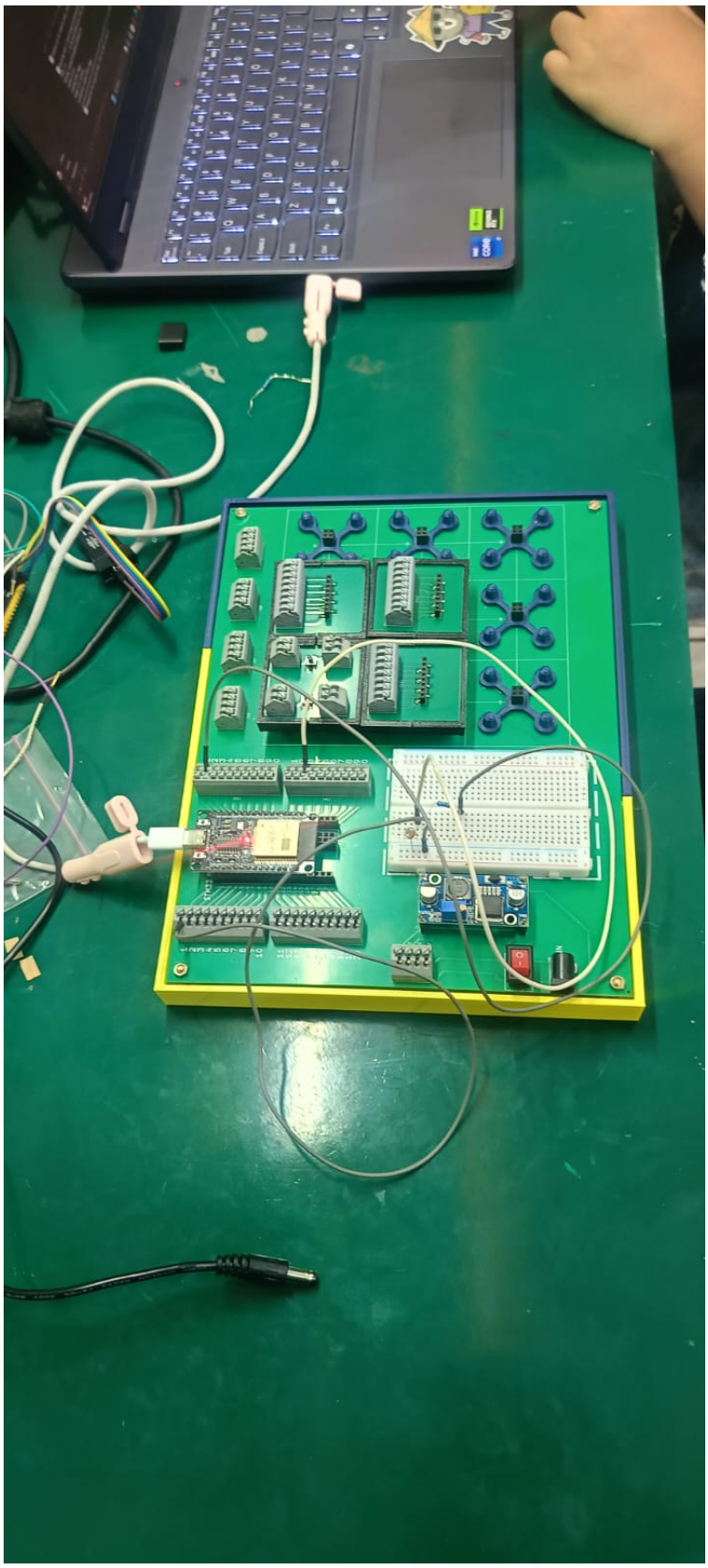


Gambar 30: Hasil Serial Monitor dengan DEBOUNCE_DELAY 200 milidetik, cuplikan 3

Dari ketiga nilai DEBOUNCE_DELAY yang diuji, semakin besar nilainya maka pembacaan semakin bersih dari getaran tombol, namun respons tombol menjadi lebih lambat. Nilai 10 milidetik terlihat belum cukup menyaring getaran karena counterNoDebounce dan counterWithDebounce naik hampir bersamaan, sedangkan nilai 200 milidetik membuat counterWithDebounce jauh lebih stabil dibanding counterNoDebounce.

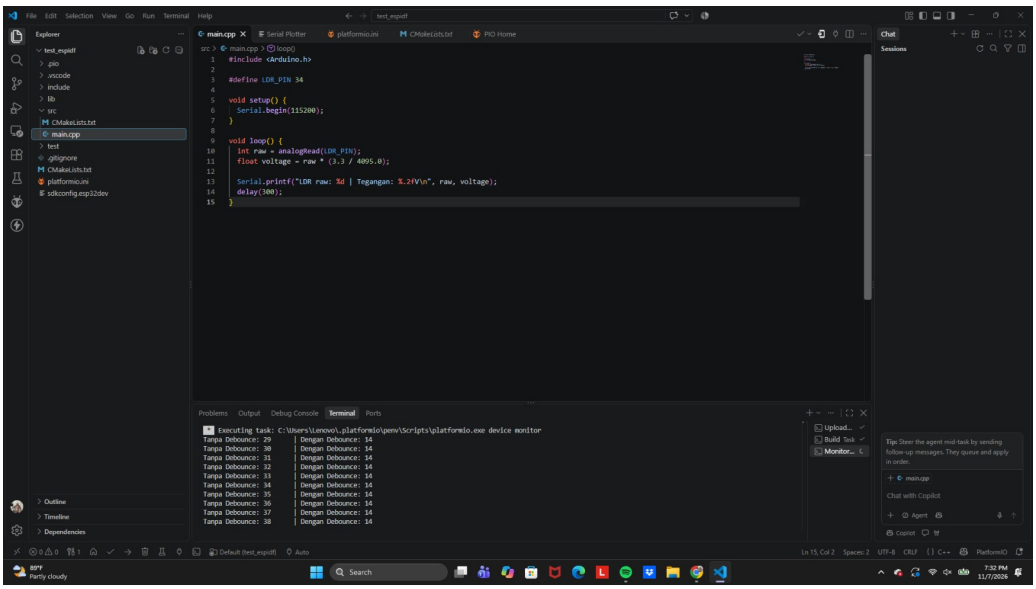
1.5 Percobaan 5: Akses GPIO Analog, Pembacaan ADC dengan LDR

LDR dirangkai sebagai pembagi tegangan bersama resistor 10kOhm, dengan titik tengah keduanya dihubungkan ke GPIO 34.



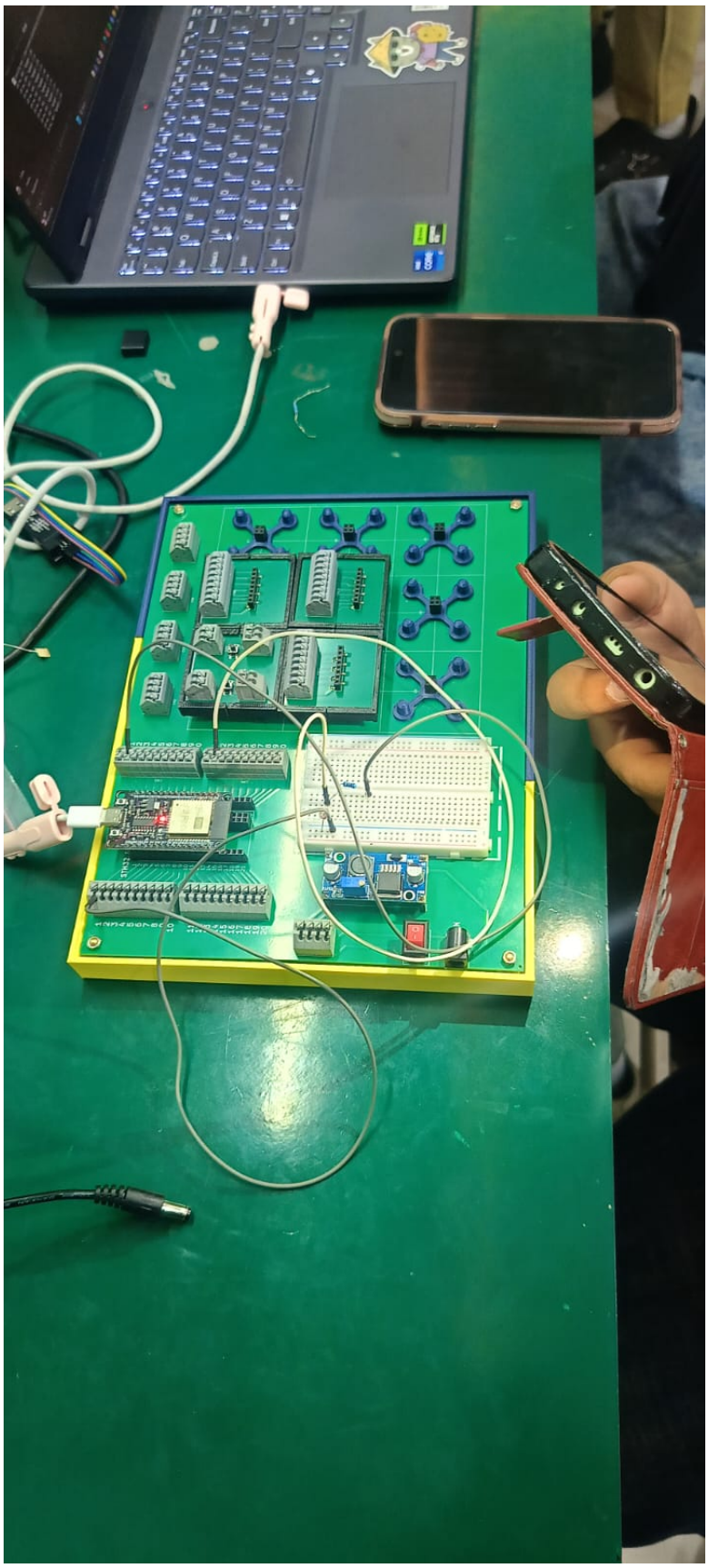
Gambar 31: Rangkaian pembagi tegangan LDR pada breadboard

Kode program ditulis untuk membaca nilai ADC mentah dan mengonversinya menjadi tegangan.

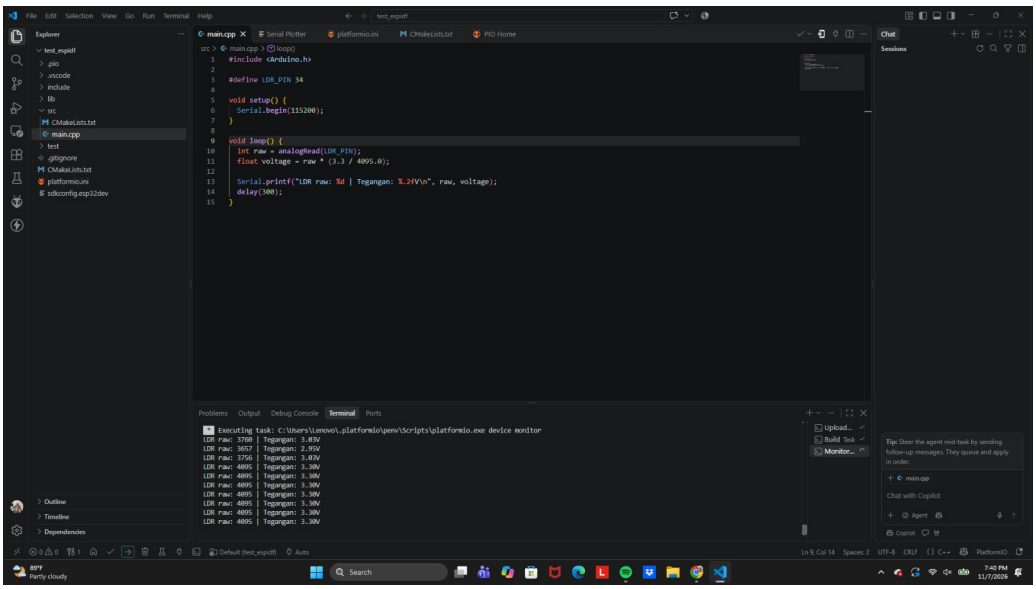


Gambar 32: Kode program pembacaan ADC menggunakan LDR

LDR disinari menggunakan cahaya lampu senter ponsel dan nilai pembacaannya diamati dari Serial Monitor.

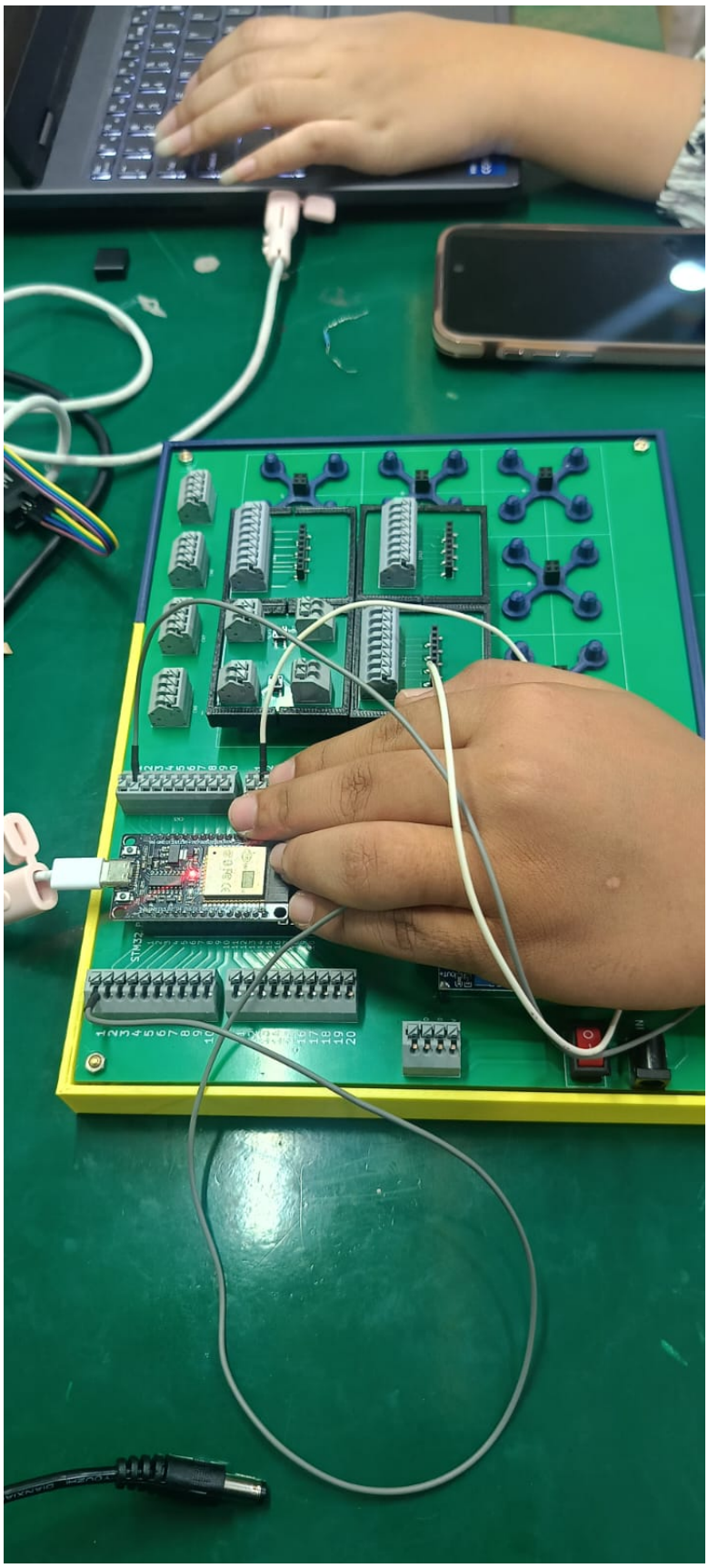


Gambar 33: LDR disinari dengan cahaya untuk menguji nilai pembacaan tinggi

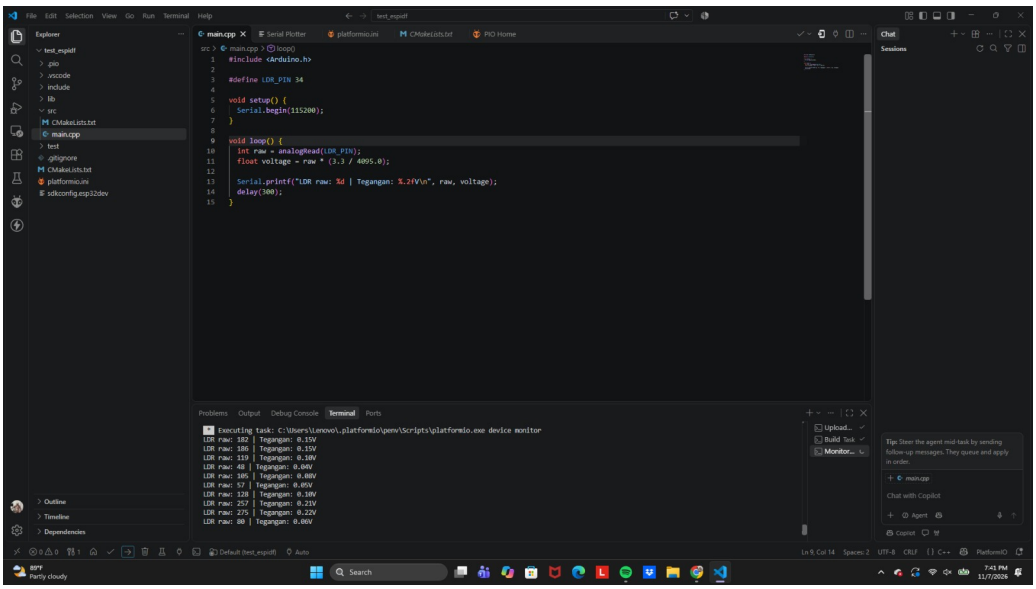


Gambar 34: Hasil pembacaan ADC saat LDR disinari, nilai mendekati 4095

LDR kemudian ditutup langsung dengan tangan dan nilai pembacaannya diamati kembali.

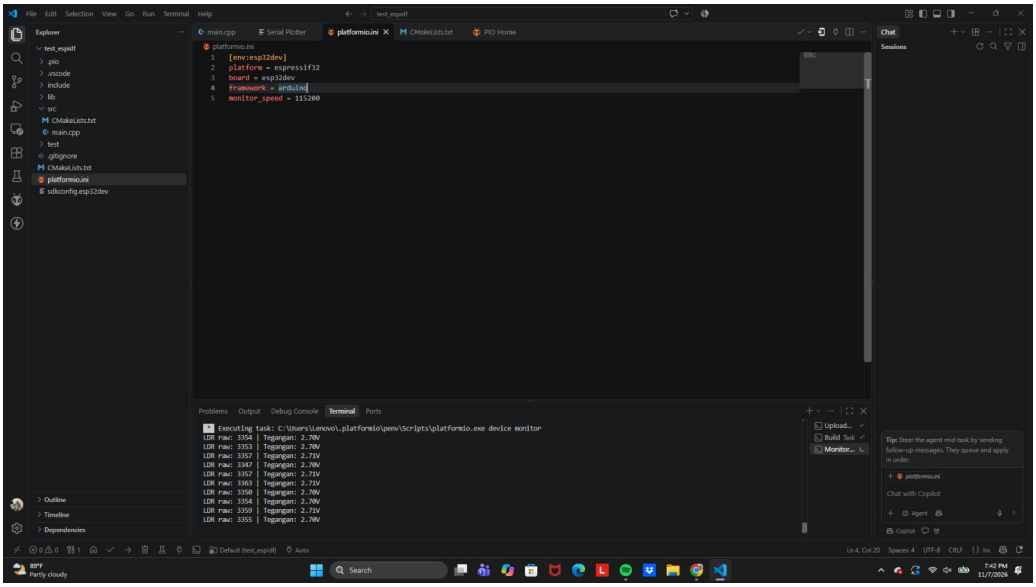


Gambar 35: LDR ditutup dengan tangan untuk menguji nilai pembacaan rendah



Gambar 36: Hasil pembacaan ADC saat LDR ditutup, nilai mendekati 0

Pada kondisi pencahayaan sedang, LDR menunjukkan nilai pembacaan di antara kedua kondisi ekstrem tersebut.



Gambar 37: Hasil pembacaan ADC pada kondisi pencahayaan sedang

Nilai

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Blink STM32 Blackpill

Program Blink berhasil dijalankan pada board STM32 Blackpill melalui ST-Link dengan status Build dan Upload SUCCESS. LED onboard pada pin PC13 berkedip nyala dan mati tiap setengah detik, sesuai yang diharapkan. Hasil ini sejalan dengan teori bahwa LED onboard Blackpill bersifat aktif LOW, sehingga nilai LOW pada digitalWrite justru membuat LED menyala, bukan sebaliknya. Percobaan ini

juga mengonfirmasi bahwa STM32 Blackpill tidak dapat diprogram langsung melalui kabel USB karena tidak memiliki chip USB-to-Serial maupun programmer onboard, sehingga wajib menggunakan ST-Link eksternal melalui jalur SWD.

2.2 Percobaan 2: Blink ESP32 dengan Framework ESP-IDF

Program Blink pada ESP32 berhasil dijalankan menggunakan framework ESP-IDF dengan titik masuk program app_main(), bukan setup() dan loop() seperti pada Arduino. Perbedaan mendasar yang teramati adalah penggunaan vTaskDelay(pdMS_TO_TICKS(500)) sebagai pengganti delay(500). Fungsi ini bekerja berbasis tick FreeRTOS dan tidak memblokir task lain, berbeda dengan delay() pada Arduino yang bersifat blocking. Selain itu, ESP-IDF memerlukan konfigurasi gpio_reset_pin() dan gpio_set_direction() secara eksplisit, sedangkan pada Arduino cukup satu baris pinMode(). Hal ini menunjukkan bahwa ESP-IDF memberi kontrol lebih rinci terhadap peripheral, namun dengan baris kode yang lebih banyak dibanding Arduino.

2.3 Percobaan 3: Pull-up dan Pull-down, Eksternal vs Internal

Pengujian pull-up eksternal pada GPIO 32 dan pull-down eksternal pada GPIO 33 menunjukkan hasil yang konsisten dengan teori, yaitu pull-up bernilai HIGH saat tombol tidak ditekan dan berubah LOW saat ditekan, sedangkan pull-down bernilai LOW saat tidak ditekan dan berubah HIGH saat ditekan. Setelah resistor eksternal dilepas dan diganti dengan INPUT_PULLUP serta INPUT_PULLDOWN internal pada pin yang sama, hasil pembacaan menunjukkan pola yang sama persis dengan kondisi eksternal. Hal ini membuktikan bahwa resistor pull internal pada ESP32 dapat menggantikan fungsi resistor eksternal sepenuhnya untuk GPIO yang mendukungnya, tanpa mengubah logika pembacaan tombol. Konsistensi hasil antara kedua metode ini juga menegaskan bahwa kondisi floating pada pin digital input harus selalu dihindari, karena nilai yang terbaca akan tidak menentu tanpa adanya resistor pull, baik eksternal maupun internal.

2.4 Percobaan 4: Debouncing pada Input Tombol GPIO

Hasil pengujian pada tiga nilai DEBOUNCE_DELAY menunjukkan perbedaan yang jelas. Pada DEBOUNCE_DELAY 10 milidetik, nilai counterNoDebounce dan counterWithDebounce naik hampir berdekatan untuk setiap tekanan tombol, menunjukkan bahwa periode 10 milidetik masih terlalu singkat untuk menyaring getaran kontak tombol secara efektif. Pada DEBOUNCE_DELAY 50 milidetik, selisih antara kedua counter mulai terlihat lebih jelas. Pada DEBOUNCE_DELAY 200 milidetik, counterWithDebounce jauh lebih stabil dan tidak banyak bertambah dibanding counterNoDebounce yang terus naik seiring waktu, menunjukkan penyaringan getaran yang jauh lebih baik.

Dari ketiga hasil tersebut terlihat trade-off yang sesuai dengan teori, yaitu semakin besar nilai DEBOUNCE_DELAY, semakin bersih pembacaan tombol dari efek bouncing, namun responsivitas tombol terhadap penekanan yang cepat dan berulang menjadi semakin lambat. Nilai delay yang terlalu kecil berisiko masih menghitung bouncing sebagai penekanan ganda, sedangkan nilai delay yang terlalu besar berisiko mengabaikan penekanan tombol yang dilakukan terlalu cepat setelah penekanan sebelumnya.

2.5 Percobaan 5: Pembacaan ADC dengan LDR

Pengujian pembacaan ADC pada LDR menunjukkan tiga kondisi berbeda. Saat LDR disinari cahaya, nilai ADC mentah yang terbaca berada pada rentang tinggi, mendekati nilai maksimum 4095, dengan tegangan mendekati 3.3 Volt. Saat LDR ditutup sehingga tidak menerima cahaya, nilai ADC mentah turun jauh ke rentang rendah, dengan tegangan mendekati 0 Volt. Pada kondisi pencahayaan sedang, nilai ADC berada di antara kedua kondisi ekstrem tersebut.

Hasil ini menunjukkan karakteristik utama sinyal analog, yaitu kemampuannya merepresentasikan banyak kondisi secara kontinu, berbeda dengan sinyal digital pada Percobaan 3 dan 4 yang hanya mengenal dua kondisi, HIGH atau LOW. Resistansi LDR berubah berdasarkan intensitas cahaya yang diterimanya, sehingga tegangan pada titik tengah rangkaian pembagi tegangan turut berubah secara proporsional, dan ADC 12-bit pada ESP32 mampu menangkap perubahan tersebut dalam 4096 tingkat nilai berbeda.

3 Hasil Tugas Modul

3.1 Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

Rancangan menggunakan metode scanning baris untuk membaca matrix pushbutton. Tiga pin digunakan sebagai baris (output, GPIO 13, 14, dan 27) dan tiga pin lain digunakan sebagai kolom (input dengan pull-up internal, GPIO 26, 25, dan 33). Setiap baris diaktifkan secara bergantian dengan memberi logika LOW, sementara baris lain diberi logika HIGH (non-aktif). Saat suatu baris aktif dan tombol pada baris tersebut ditekan, kolom yang terhubung akan terbaca LOW. Hasil pembacaan seluruh tombol disimpan pada array `raw[3][3]` oleh fungsi `scanMatrix()`. Matrix LED pada rancangan ini menggunakan sembilan pin individual, masing-masing mewakili satu posisi LED pada baris dan kolom yang sama dengan posisi tombolnya.

Debouncing diterapkan pada setiap tombol secara terpisah, mengacu pada teknik yang sama seperti Percobaan 4, yaitu menyimpan `lastReading`, `stableState`, dan `lastChange` untuk masing-masing dari sembilan tombol. LED pada posisi (baris, kolom) tertentu hanya berubah status setelah pembacaan tombol pada posisi yang sama benar-benar stabil selama periode `debounceDelay` sebesar 50 ms, bukan langsung mengikuti sinyal mentah yang masih mungkin bouncing. Setiap penekanan tombol yang sudah stabil membalik (toggle) status LED pada posisi yang sama, sehingga penekanan pertama menyalakan LED dan penekanan berikutnya mematikannya.

Listing 1: Kode program Matrix Pushbutton 3x3 mengendalikan Matrix LED 3x3 dengan debouncing per tombol

```
#include <Arduino.h>

const int rowPins[3] = {13, 14, 27};
const int colPins[3] = {26, 25, 33};
const int ledPins[3][3] = {
  {16, 17, 18},
  {19, 21, 22},
  {23, 32, 4}
};

const unsigned long debounceDelay = 50;

bool lastReading[3][3];
```

```
bool stableState[3][3];
unsigned long lastChange[3][3];
bool ledOn[3][3];

void scanMatrix(bool raw[3][3]) {
  for (int r = 0; r < 3; r++) {
    for (int i = 0; i < 3; i++) digitalWrite(rowPins[i], HIGH);
    digitalWrite(rowPins[r], LOW);
    delayMicroseconds(10);
    for (int c = 0; c < 3; c++) {
      raw[r][c] = (digitalRead(colPins[c]) == LOW);
    }
  }
  for (int i = 0; i < 3; i++) digitalWrite(rowPins[i], HIGH);
}

void setup() {
  Serial.begin(115200);

  for (int i = 0; i < 3; i++) {
    pinMode(rowPins[i], OUTPUT);
    digitalWrite(rowPins[i], HIGH);
    pinMode(colPins[i], INPUT_PULLUP);
  }

  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      pinMode(ledPins[r][c], OUTPUT);
      digitalWrite(ledPins[r][c], LOW);
      lastReading[r][c] = false;
      stableState[r][c] = false;
      lastChange[r][c] = 0;
      ledOn[r][c] = false;
    }
  }

  Serial.println("Matrix Pushbutton 3x3 -> Matrix LED 3x3");
}

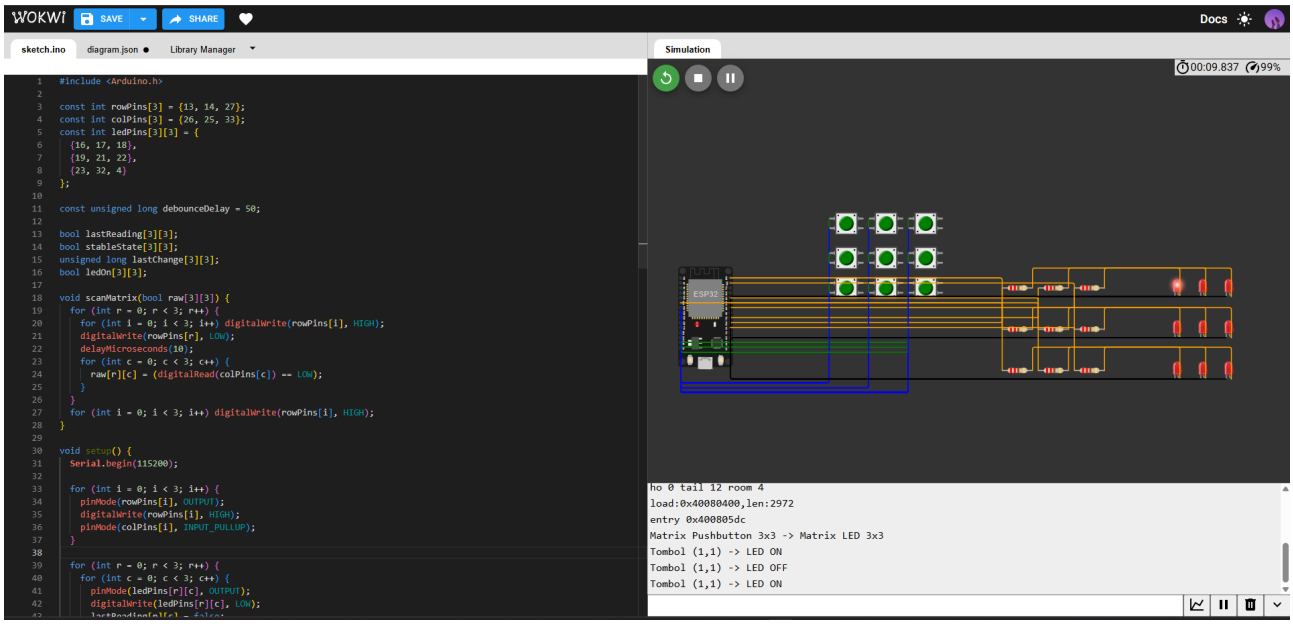
void loop() {
  bool raw[3][3];
  scanMatrix(raw);
  unsigned long now = millis();

  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      if (raw[r][c] != lastReading[r][c]) {
        lastChange[r][c] = now;
        lastReading[r][c] = raw[r][c];
      }

      if ((now - lastChange[r][c]) > debounceDelay &&
          raw[r][c] != stableState[r][c]) {
        stableState[r][c] = raw[r][c];
```

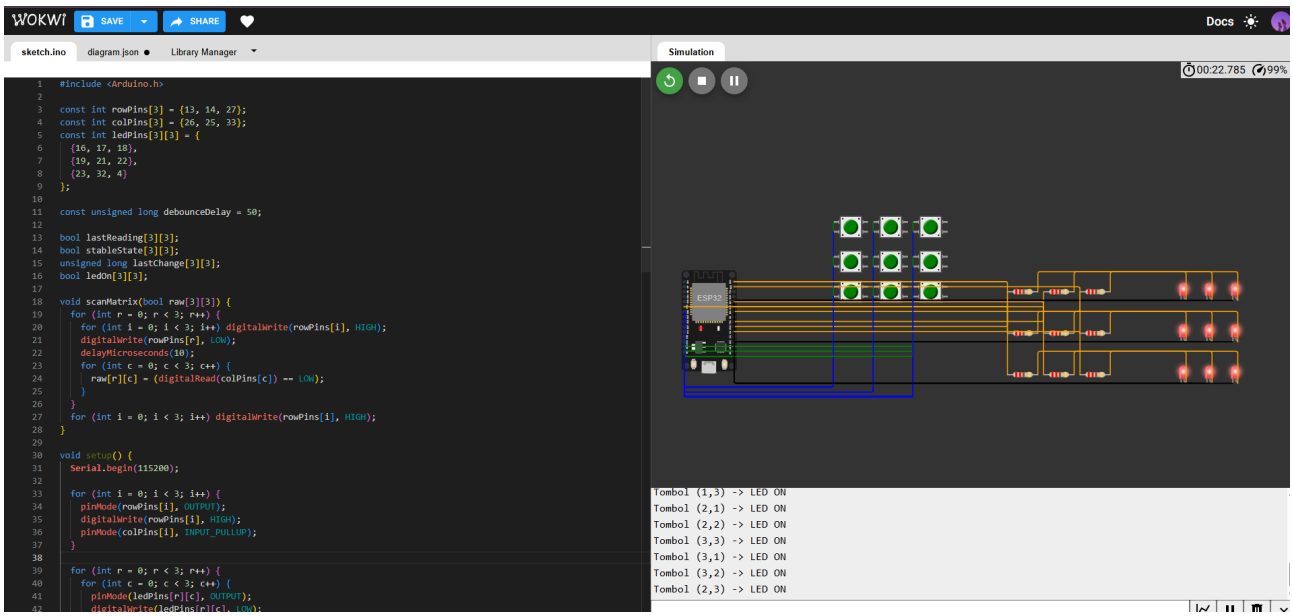
```
68
69          if (stableState[r][c]) {
70            ledOn[r][c] = !ledOn[r][c];
71            digitalWrite(ledPins[r][c], ledOn[r][c] ? HIGH : LOW);
72            Serial.printf("Tombol (%d,%d) -> LED %s\n",
73                          r + 1, c + 1, ledOn[r][c] ? "ON" : "OFF");
74          }
75        }
76      }
77    }
78  }
```

Gambar 38 menunjukkan kondisi saat hanya tombol baris 1 kolom 1 yang ditekan. Hanya LED baris 1 kolom 1 yang menyala, dan Serial Monitor mencatat `Tombol (1,1) -> LED ON`. Hal ini membuktikan bahwa tombol pada posisi tertentu mengendalikan LED pada posisi yang sama.



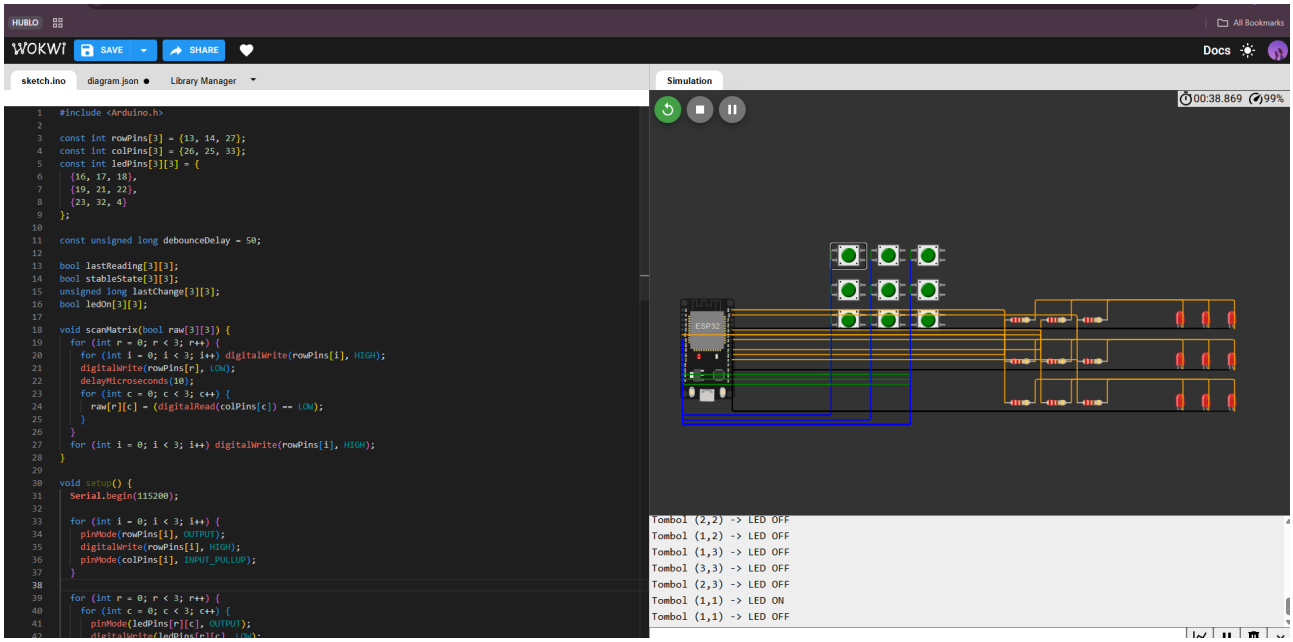
Gambar 38: Tombol (1,1) ditekan sehingga hanya LED baris 1 kolom 1 yang menyala

Gambar 39 menunjukkan kondisi setelah seluruh tombol ditekan satu per satu. Kesembilan LED menyala dan Serial Monitor mencatat `LED ON` untuk setiap tombol, sehingga seluruh tombol pada matriks terbukti bekerja.



Gambar 39: Seluruh tombol ditekan sehingga kesembilan LED menyala

Gambar 40 menunjukkan kondisi setelah seluruh tombol ditekan kembali. Status setiap LED dibalik (toggle) sehingga seluruh LED mati, dan Serial Monitor mencatat `LED OFF` untuk setiap tombol. Satu kali penekanan hanya menghasilkan satu baris log, yang menunjukkan debouncing bekerja dengan baik.



Gambar 40: Seluruh tombol ditekan kembali sehingga kesembilan LED mati

3.2 Pertanyaan Analisis

3.2.1 Pertanyaan 1

Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing.

Debouncing tetap diperlukan pada rangkaian matrix pushbutton ini meskipun pada satu waktu biasanya hanya satu tombol yang ditekan, karena penyebab bouncing bukan berasal dari banyaknya tombol yang ditekan bersamaan, melainkan dari sifat mekanik kontak logam pada tombol itu sendiri, sebagaimana dijelaskan pada bagian Dasar Teori mengenai Debouncing. Setiap kali satu tombol ditekan atau dilepas, kontak logam di dalamnya tetap memantul dan bergetar dalam waktu sangat singkat sebelum benar-benar diam, sehingga menghasilkan beberapa transisi sinyal HIGH-LOW meskipun hanya ada satu aksi tekan. Pada rangkaian matrix, proses scanning baris dilakukan berulang dengan sangat cepat, sehingga getaran sinyal pada satu tombol akan ikut terbaca berkali-kali pada siklus scanning yang berbeda jika tidak disaring. Karena program ini membalik (toggle) status LED pada setiap penekanan yang terbaca, tanpa debouncing satu kali tekanan dapat terbaca sebagai beberapa penekanan sehingga LED berkedip-kedip atau berakhir pada status yang salah, karena pembacaan kolom mengikuti sinyal mentah yang belum stabil.

3.2.2 Pertanyaan 2

Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?

Rancangan ini menggunakan pull-up internal pada pin kolom melalui INPUT_PULLUP. Saat tombol tidak ditekan, pin kolom tertarik ke VCC melalui resistor pull-up internal sehingga kondisinya terbaca HIGH. Saat baris yang bersangkutan diaktifkan dengan logika LOW dan tombol pada perpotongan baris-kolom tersebut ditekan, pin kolom akan terhubung ke logika LOW pada baris tersebut, sehingga kondisinya berubah menjadi LOW. Kondisi pin kolom tidak boleh dibiarkan floating karena tanpa resistor pull, baik eksternal maupun internal, nilai tegangan pada pin tidak memiliki referensi pasti saat tombol tidak ditekan. Pin yang floating sangat rentan terhadap noise dari lingkungan sekitar, sehingga nilai HIGH atau LOW yang terbaca bisa berubah-ubah secara acak tanpa ada tombol yang ditekan sama sekali, yang pada rangkaian matrix ini dapat menyebabkan status LED berubah sendiri pada posisi yang sebenarnya tidak ditekan.

3.2.3 Pertanyaan 3

Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

Jika salah satu tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC seperti pada Percobaan 5, perbedaan mendasarnya terletak pada jumlah kondisi yang dapat direpresentasikan oleh pembacaan tersebut. Pembacaan digital input pada tombol hanya mengenal dua kondisi, yaitu HIGH atau LOW, karena digitalRead hanya membandingkan tegangan terhadap satu ambang batas tertentu untuk menentukan apakah pin berada pada level tinggi atau rendah. Sementara itu, pembacaan analog input melalui ADC pada LDR menghasilkan nilai digital bertingkat, pada ESP32 berupa rentang 0 sampai 4095 untuk resolusi 12-bit, karena ADC melakukan proses sampling dan kuantisasi terhadap tegangan yang berubah secara kontinu sesuai intensitas cahaya yang diterima LDR. ADC dapat merepresentasikan lebih dari dua kondisi karena pada dasarnya ADC membagi rentang tegangan referensi menjadi banyak level diskrit, bukan hanya dua level seperti pada pembacaan

digital, sehingga perubahan tegangan yang kecil sekalipun tetap dapat terdeteksi sebagai perubahan nilai, bukan hanya sebagai dua kondisi tetap.

4 Kesimpulan

Praktikum Modul 1 ini bertujuan untuk mengenalkan arsitektur board STM32 Blackpill dan ESP32, membedakan karakteristik framework Arduino dan ESP-IDF, serta mengimplementasikan pembacaan GPIO dalam tiga mode akses, yaitu digital output, digital input, dan analog input. Berdasarkan seluruh percobaan yang telah dilakukan, tujuan tersebut telah tercapai dengan hasil yang sesuai dengan teori yang dipelajari.

Pada Percobaan 1 dan 2, program Blink berhasil dijalankan pada kedua board. STM32 Blackpill memerlukan ST-Link eksternal untuk flashing karena tidak memiliki chip USB-to-Serial maupun programmer onboard, sedangkan ESP32 dapat diprogram langsung melalui kabel USB karena sudah dilengkapi chip USB-to-Serial bawaan. Perbedaan framework juga teramati jelas, Arduino menggunakan struktur setup() dan loop() yang sederhana, sedangkan ESP-IDF menggunakan app_main() sebagai titik masuk program dengan fungsi-fungsi yang lebih native terhadap hardware, seperti gpio_set_direction() dan vTaskDelay() berbasis FreeRTOS.

Pada Percobaan 3, pengujian pull-up dan pull-down eksternal maupun internal menunjukkan hasil yang konsisten dan sesuai teori, yaitu pull-up bernilai HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan pull-down berlaku sebaliknya. Hasil ini membuktikan bahwa resistor pull internal pada ESP32 dapat menggantikan resistor eksternal tanpa mengubah logika pembacaan, selama pin GPIO yang digunakan memang mendukung fitur tersebut.

Pada Percobaan 4, pengujian debouncing dengan tiga nilai DEBOUNCE_DELAY yang berbeda, yaitu 10, 50, dan 200 milidetik, menunjukkan adanya trade-off antara kebersihan sinyal dan responsivitas tombol. Semakin besar nilai delay, semakin bersih hasil pembacaan dari efek bouncing, namun semakin lambat pula tombol merespons penekanan yang cepat dan berulang. Hal ini menjadi pembelajaran penting bahwa pemilihan nilai debounce harus disesuaikan dengan kebutuhan aplikasi.

Pada Percobaan 5, pembacaan ADC menggunakan LDR berhasil menunjukkan karakteristik sinyal analog yang mampu merepresentasikan banyak kondisi secara kontinu, berbeda dengan sinyal digital pada percobaan sebelumnya yang hanya mengenal dua kondisi. Nilai ADC berubah sesuai intensitas cahaya yang diterima LDR, membuktikan bahwa ADC bekerja melalui proses sampling dan kuantisasi tegangan menjadi level-level diskrit.

5 Kesimpulan

Dari praktikum Modul 1 ini, dapat disimpulkan bahwa STM32 Blackpill memerlukan ST-Link untuk flashing karena tidak memiliki USB-to-Serial onboard, sedangkan ESP32 dapat diprogram langsung lewat USB. Framework Arduino lebih sederhana dengan setup() dan loop(), sedangkan ESP-IDF menggunakan app_main() dengan kontrol yang lebih mendalam terhadap hardware. Pengujian pull-up dan pull-down, baik eksternal maupun internal, menghasilkan logika pembacaan yang sama, dan pin digital input tidak boleh dibiarkan floating agar pembacaannya tidak acak. Debouncing terbukti diperlukan untuk menyaring getaran sinyal tombol, dengan nilai delay yang lebih besar memberikan pembacaan lebih bersih namun respons lebih lambat. Pembacaan ADC pada LDR menunjukkan bahwa sinyal analog dapat merepresentasikan banyak kondisi, berbeda dengan sinyal digital yang

hanya HIGH atau LOW. Seluruh hasil percobaan sesuai dengan teori yang dipelajari dan menjadi dasar penting sebelum masuk ke materi interfacing sensor dan aktuator pada modul selanjutnya.

6 Lampiran

6.1 Dokumentasi saat praktikum

